BIANCA GOMES DANTAS DE ALMEIDA
GIOVANNA GOMES SILVA
JAILTON SANTOS DE MATOS
LUCAS ALVES DE SOUZA
LUCAS NEVES DOS SANTOS

# Gerenciador de Eventos Acadêmicos

Versão 1.0

2026

# 1. Visão Geral do Projeto

## 1.1 Descrição Executiva

O Gerenciador de Eventos Acadêmicos é uma aplicação web destinada à organização e ao acompanhamento de eventos promovidos no ambiente acadêmico, como palestras, workshops, semanas temáticas e congressos internos. O sistema cobre o ciclo completo da atividade, desde a criação e a divulgação do evento pelo organizador até a inscrição dos participantes, o registro de presença, a emissão do certificado digital e a coleta de avaliações posteriores ao encerramento.

A motivação do projeto parte de uma dificuldade recorrente na gestão de eventos acadêmicos: o controle disperso em planilhas, formulários avulsos e mensagens informais, que compromete a confiabilidade das listas de presença e torna a emissão de certificados um processo manual e sujeito a erros. A proposta é centralizar essas etapas em uma única plataforma, na qual cada certificado emitido esteja obrigatoriamente vinculado a um registro de presença verificável por meio de código de autenticidade.

O diferencial técnico da solução é a incorporação de inteligência artificial à etapa de cadastro. Em vez de preencher formulários extensos, o organizador informa os dados do evento em texto livre e o sistema extrai automaticamente os campos estruturados, submetendo-os à confirmação humana antes da persistência. A arquitetura prevê que a indisponibilidade desse recurso não interrompa a operação, mantendo o preenchimento manual sempre disponível como alternativa.

Espera-se, como resultado, a redução do tempo despendido na criação e na administração de eventos, maior confiabilidade dos registros de participação e a entrega imediata de certificados aos participantes, sem necessidade de intervenção manual da organização.

## 1.2 Objetivos e Escopo

### 1.2.1 Objetivo Geral

Desenvolver uma aplicação web para a gestão completa de eventos acadêmicos, capaz de administrar inscrições com controle de vagas e fila de espera, registrar a presença dos participantes e emitir certificados digitais com validação pública, incorporando recursos de inteligência artificial ao processo de cadastro.

### 1.2.2 Objetivos Específicos

A. Disponibilizar autenticação e controle de acesso diferenciado para participantes e organizadores;

B. Permitir a criação e a divulgação de eventos, tanto por preenchimento manual quanto por extração automática de dados a partir de texto livre;

C. Controlar a ocupação das vagas, incluindo a formação de fila de espera e a promoção automática em caso de desistência;

D. Registrar a presença dos participantes durante a realização do evento;

E. Emitir certificados digitais em formato PDF, vinculados à presença e dotados de código de autenticidade verificável publicamente;

F. Coletar avaliações após o encerramento do evento e sintetizá-las automaticamente em um resumo consolidado;

G. Assegurar a continuidade da operação em caso de indisponibilidade do serviço de inteligência artificial.

### 1.2.3 Escopo do Projeto

O escopo compreende o desenvolvimento do servidor de aplicação, da base de dados relacional e da interface web de acesso, abrangendo os módulos de autenticação, cadastro de eventos assistido por inteligência artificial, inscrições e fila de espera, registro de presença, certificação e avaliação. Inclui ainda a integração com um serviço de modelo de linguagem para a extração de dados e a geração de resumos, bem como a geração dos certificados em PDF.

Considera-se o projeto concluído quando for possível executar, de ponta a ponta, o fluxo que vai da criação de um evento pelo organizador até a emissão e a validação pública do certificado de um participante que teve presença registrada.

### 1.2.4 Limitações de Escopo

Não fazem parte desta versão do sistema as seguintes funcionalidades, por não serem essenciais à validação da proposta e por ampliarem significativamente a complexidade do desenvolvimento:

a) Cobrança, pagamento ou emissão de documentos fiscais para eventos pagos;

b) Divisão do evento em sessões ou palestras independentes, com grade de horários e presença individual por sessão;

c) Submissão e avaliação de trabalhos acadêmicos, bem como o perfil de avaliador correspondente;

d) Envio automático de notificações por correio eletrônico ou mensageria;

e) Aplicativo móvel nativo, sendo o acesso realizado por navegador com interface responsiva.

## 1.3 Regras de Negócio

As regras de negócio definem as restrições e os comportamentos obrigatórios do sistema, independentemente da tecnologia empregada na implementação. Cada regra recebe um identificador próprio e está vinculada aos requisitos funcionais que a originaram, permitindo o rastreamento entre a especificação, o código e os casos de teste.

RN-01: Ao realizar a inscrição, caso o número de inscrições confirmadas seja inferior ao limite de vagas, a inscrição é confirmada imediatamente; caso contrário, o participante é alocado em fila de espera, recebendo a próxima posição disponível (RF-05 e RF-06).

RN-02: O cancelamento de uma inscrição confirmada promove automaticamente o primeiro participante da fila de espera, na mesma operação. O cancelamento de uma inscrição que já se encontra em fila apenas reordena as posições subsequentes (RF-06 e RF-07).

RN-03: Um mesmo usuário não pode manter duas inscrições ativas no mesmo evento. A reinscrição após cancelamento é permitida e aloca o participante ao final da fila (RF-05).

RN-04: O registro de presença somente é aceito durante o período de realização do evento, acrescido de uma tolerância definida em configuração, e exclusivamente pelo organizador responsável pelo evento, para inscrições em situação confirmada (RF-08).

RN-05: O certificado somente é emitido mediante a existência de registro de presença para a inscrição correspondente. A cada certificado é atribuído um código único de

autenticidade, e novas solicitações de emissão retornam o mesmo documento já gerado (RF-09).

RN-06: A avaliação do evento somente é aceita após o encerramento da atividade e para participantes com presença registrada, admitindo-se uma única avaliação por inscrição (RF-10).

RN-07: O resumo automático das avaliações é gerado apenas quando houver quantidade mínima de avaliações recebidas, definida em configuração. A regeração substitui o resumo anterior (RF-10).

RN-08: Somente o organizador criador do evento pode editá-lo, publicá-lo, cancelá-lo, registrar presenças e consultar a lista de inscritos (RNF-04).

RN-09: Eventos encerrados ou cancelados não aceitam edição nem novas inscrições. A inscrição somente é permitida em eventos publicados.

RN-10: A indisponibilidade do serviço de inteligência artificial não impede o cadastro do evento. Nessa situação, o sistema informa a falha e disponibiliza o preenchimento manual, preservando as demais regras de negócio (RNF-02).

RN-11: As senhas devem possuir extensão mínima definida em configuração e são armazenadas exclusivamente sob a forma de resumo criptográfico, nunca em texto puro (RNF-03).

# 2. Arquitetura do Sistema

## 2.1 Componentes Principais

O sistema é organizado em módulos que cobrem todo o ciclo de vida do evento. O Módulo de Autenticação controla o acesso de Participantes e Organizadores, definindo as permissões de cada perfil. O Módulo de Atendimento e Cadastro por IA substitui os formulários tradicionais pela interpretação de texto em linguagem natural, permitindo que o organizador crie eventos a partir de um texto colado e que os participantes esclareçam dúvidas sobre as atividades disponíveis.

O Módulo de Inscrições e Presença administra a ocupação das vagas, a fila de espera e o registro de presença dos participantes durante a realização do evento. Por fim, o Módulo de Certificação valida se o participante cumpriu os requisitos de participação e gera automaticamente o certificado final em PDF, com código de autenticidade destinado à verificação pública.

A representação gráfica desses módulos e de suas dependências encontra-se no Diagrama de Componentes, apresentado na seção 3.11.

## 2.2 Decisões Arquiteturais (ADR)

Para manter o projeto simples e viável nesta primeira etapa, adotou-se uma arquitetura de aplicação única (monolito) dividida em camadas claras. A comunicação entre a interface e o servidor Java é realizada por mensagens padronizadas no formato JSON, seguindo o estilo arquitetural REST.

Para viabilizar a criação de cadastros a partir de texto livre, definiu-se o uso do recurso de saída estruturada do Spring AI nas chamadas ao modelo de linguagem. O framework anexa à requisição o esquema do objeto esperado e converte a resposta diretamente em um registro Java, de modo que a inteligência artificial devolva JSON em vez de texto corrido. O backend valida esse retorno e grava as informações no banco de dados somente após a confirmação explícita do usuário. A conversão das datas é realizada no código da aplicação, e não delegada ao desserializador, para que um campo em formato inesperado seja relatado como não identificado em vez de invalidar a extração inteira.

Optou-se, ainda, pela separação do frontend e do backend em implantações independentes. Essa decisão permite que cada parte seja publicada em uma plataforma especializada e evoluída isoladamente, ao custo da necessidade de configuração explícita de política de compartilhamento de recursos entre origens distintas e de declaração do domínio do servidor na política de segurança de conteúdo da interface.

## 2.3 Stack Tecnológica

A base do servidor foi desenvolvida em Java 21 (LTS) com o framework Spring Boot 4.1.1, cuja organização modular exige a declaração individual dos componentes de infraestrutura utilizados. O controle de acesso emprega o Spring Security 7, e a integração com o modelo de linguagem é feita pelo framework Spring AI 2.0.1. A troca de provedor de inteligência artificial é resolvida exclusivamente por configuração, sem alteração de código: o projeto mantém quatro perfis — Groq, adotado em produção, OpenRouter, OpenAI e Ollama em execução local — todos consumindo a mesma interface compatível com a da OpenAI.

Os dados de cadastros, interações com a inteligência artificial, inscrições e presenças ficam armazenados no banco de dados relacional PostgreSQL, executado em contêiner durante

o desenvolvimento e em serviço gerenciado na Neon em produção. A biblioteca iText 9.8 é utilizada diretamente no código Java para a montagem dos certificados digitais em PDF, gerados em memória a cada solicitação, sem gravação em disco. A interface do usuário é construída como aplicação de página única em React 19 com TypeScript, estilizada com Tailwind CSS 4 e empacotada pelo Vite 7, consumindo exclusivamente a interface de programação do servidor.

# 3. Especificação Técnica

## 3.1 Requisitos Funcionais

RF-01: Criar conta e fazer login (Visitante)

RF-02: Criar evento manualmente, com fotos, datas e vagas (Organizador)

RF-03: Criar evento colando um texto, com preenchimento por IA (Organizador)

RF-04: Pesquisar e filtrar eventos (Participante)

RF-05: Garantir vaga em um evento (Participante)

RF-06: Entrar na fila de espera e subir na fila caso haja desistência (Sistema)

RF-07: Cancelar a própria inscrição (Participante)

RF-08: Marcar presença/check-in na hora do evento (Organizador)

RF-09: Gerar certificado para quem compareceu ao evento (Sistema)

RF-10: Enviar opinião pós-evento e gerar resumo das avaliações (Participante e Sistema)

**Figura 1 — Diagrama de Casos de Uso: Visitante e Participante.** Funcionalidades acessíveis ao visitante e ao participante, que herda os casos do visitante. Os relacionamentos «include» tornam obrigatórias a alocação de vaga na inscrição (RN-01) e a verificação de presença na emissão do certificado (RN-05); o «extend» indica que a promoção do primeiro da fila só ocorre quando um cancelamento libera vaga confirmada (RN-02).

**Figura 2 — Diagrama de Casos de Uso: Organizador.** Funcionalidades do organizador. O preenchimento por IA estende a criação de evento (RF-03) e depende do serviço de linguagem, ator secundário; sem ele, a criação manual continua disponível (RN-10). Toda ação sobre um evento exige que o usuário seja o seu criador (RN-08).

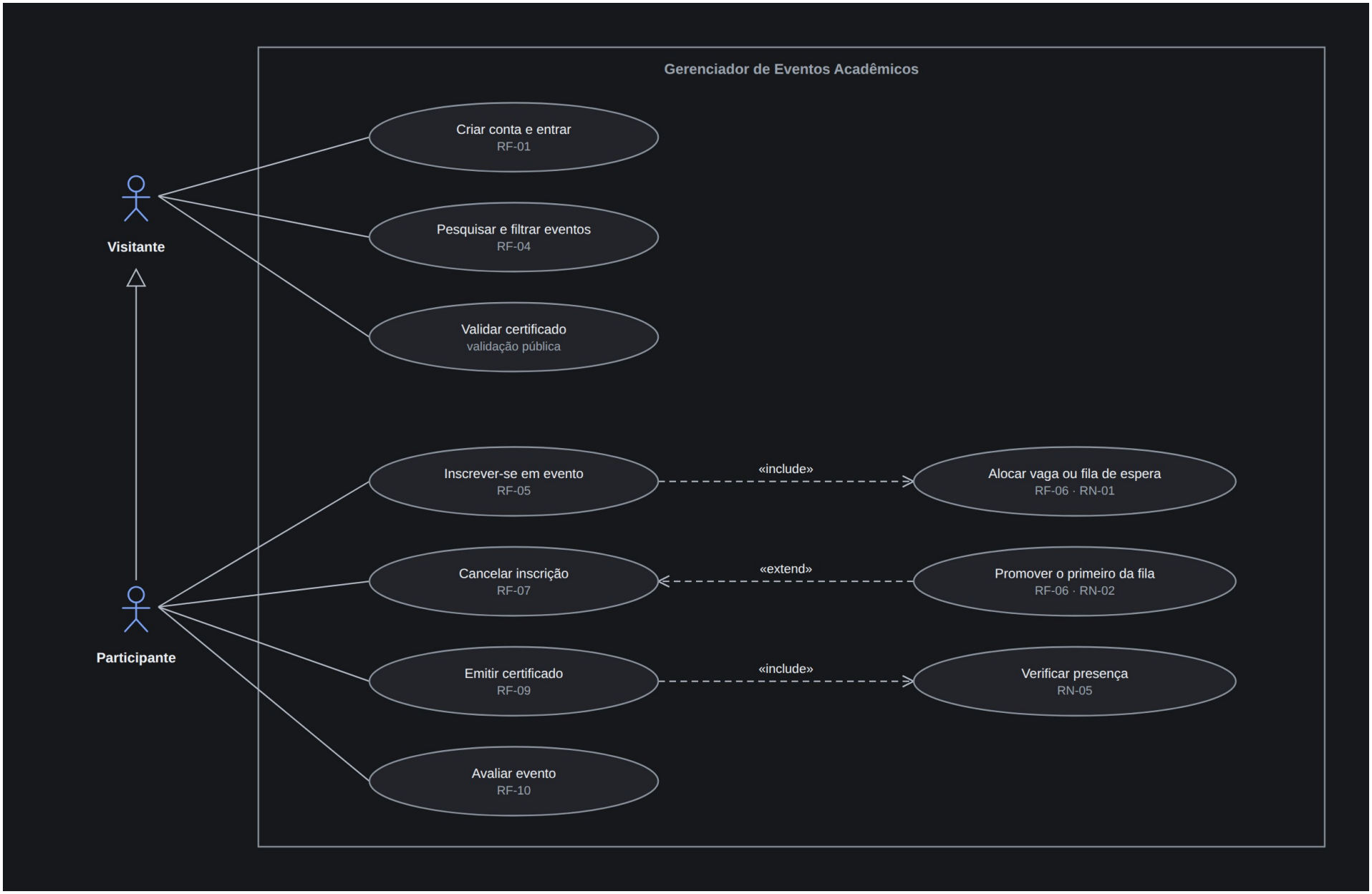



**Figura 1 — Diagrama de Casos de Uso: Visitante e Participante**

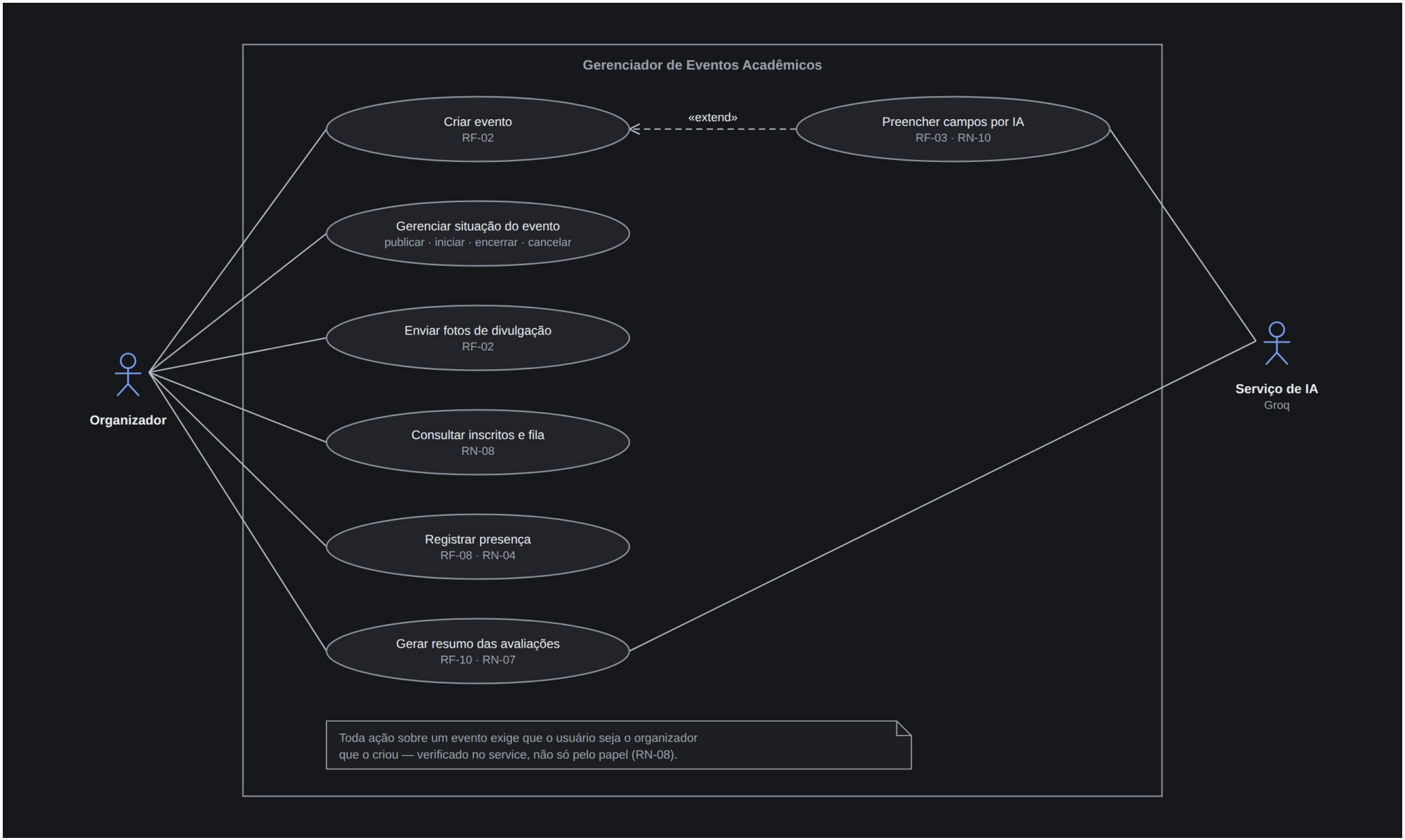



**Figura 2 — Diagrama de Casos de Uso: Organizador**

## 3.2 Requisitos Não-Funcionais

RNF-01: A lista de eventos deve carregar em menos de 2 segundos mesmo com 500 eventos cadastrados (Desempenho)

RNF-02: Se a inteligência artificial ficar indisponível, o usuário ainda consegue cadastrar o evento manualmente (Resiliência)

RNF-03: Senhas devem ter hash e as chaves de API nunca devem ser expostas no navegador (Segurança)

RNF-04: Apenas o criador do evento pode gerenciar as ações e opções do seu evento (Autorização)

RNF-05: O site deve funcionar corretamente em telas de celulares a partir de 360px de largura (Usabilidade)

## 3.3 Protótipo Baixa Fidelidade e Alta Fidelidade

A prototipação da interface foi conduzida em duas etapas sucessivas. A primeira, de baixa fidelidade, teve por finalidade definir a estrutura das telas, a hierarquia das informações e o posicionamento dos controles, sem qualquer preocupação com cores, tipografia ou identidade visual. A segunda, de alta fidelidade, materializou essas decisões na interface efetivamente implementada.

### 3.3.1 Protótipo de Baixa Fidelidade

Os wireframes a seguir representam as duas telas centrais do sistema. A primeira corresponde à listagem pública de eventos, estruturada em três blocos verticais: navegação principal, painel de filtros e lista de resultados em cartões. A segunda corresponde ao cadastro de evento pelo organizador, dividido em dois blocos distintos: a área de entrada de texto livre destinada à interpretação por inteligência artificial e, abaixo dela, o formulário revisável com os campos extraídos.

A separação visual entre esses dois blocos foi uma decisão deliberada de projeto. Ela comunica ao organizador que a extração automática produz uma sugestão, e não um cadastro concluído, reforçando no próprio desenho da tela a exigência de confirmação humana prevista na regra RN-10.

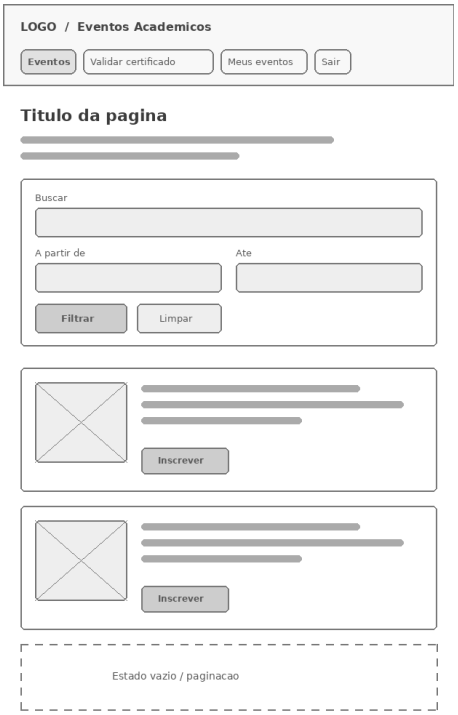



*Figura 3 — Wireframe da listagem de eventos*

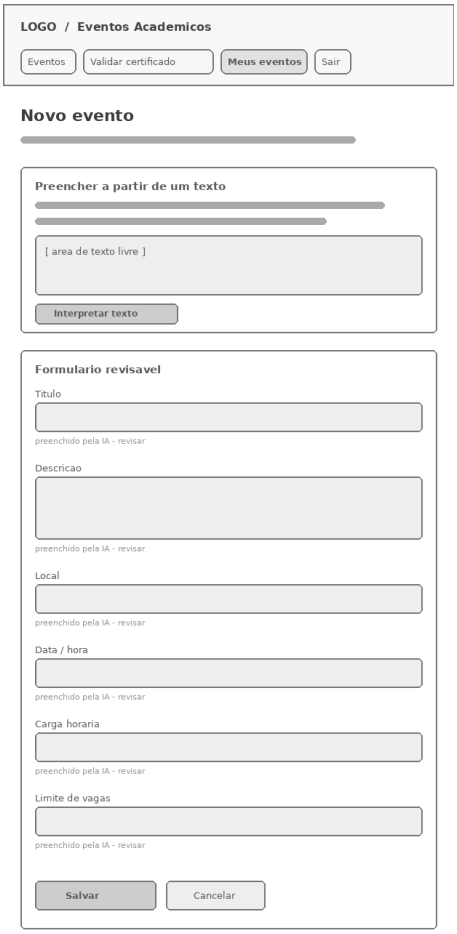



*Figura 4 — Wireframe do cadastro de evento*

### 3.3.2 Protótipo de Alta Fidelidade

As telas abaixo apresentam a interface implementada, já com a identidade visual aplicada e em execução sobre a API do sistema. A primeira exibe a listagem de eventos com o painel de filtros por texto e período, além do estado vazio exibido quando nenhum evento atende aos critérios informados. A segunda exibe o cadastro de evento, no qual o organizador cola a descrição em linguagem natural, aciona a interpretação e recebe os campos preenchidos com a indicação explícita de que o conteúdo foi gerado automaticamente e deve ser conferido antes de salvar.




*Figura 5 — Listagem de eventos (interface implementada)*




*Figura 6 — Cadastro assistido por IA (interface implementada)*

O layout adota disposição em coluna única e elementos dimensionados para toque, atendendo ao requisito RNF-05, que estabelece o funcionamento adequado em telas a partir de 360 pixels de largura. A mensagem auxiliar exibida sob os campos preenchidos automaticamente — indicando que o conteúdo foi gerado pela inteligência artificial e deve ser conferido — materializa na interface a exigência de revisão humana estabelecida nas regras de negócio.

## 3.4 Métricas e Cronograma

### 3.4.1 Cronograma

O desenvolvimento do projeto foi organizado em oito etapas sequenciais, distribuídas entre os meses de setembro e dezembro de 2026. As quatro primeiras etapas concentram as atividades de levantamento, modelagem e documentação, culminando na entrega final deste documento em 23 de outubro de 2026. As etapas seguintes destinam-se à implementação, aos testes e à apresentação do sistema.

| Etapa | Período | Entregável | Situação |
|---|---|---|---|
| 1. Levantamento de requisitos e definição de escopo | 01/09 a 15/09/2026 | Lista de requisitos funcionais e não funcionais | Concluída |
| 2. Modelagem UML | 16/09 a 05/10/2026 | Casos de uso, classes, | Concluída |

| Etapa | Período | Entregável | Situação |
|---|---|---|---|
| | | estados, atividades, interação e componentes | |
| 3. Modelagem de dados | 06/10 a 15/10/2026 | Modelo lógico, criação física e dicionário de dados | Concluída |
| 4. Prototipação e consolidação da documentação | 16/10 a 23/10/2026 | Protótipos e documentação final do software | Em andamento |
| 5. Configuração do ambiente e do pipeline | 24/10 a 02/11/2026 | Repositório, migrações, integração contínua e ambientes publicados | Prevista |
| 6. Implementação do núcleo | 03/11 a 23/11/2026 | Autenticação, eventos, inscrições e fila de espera | Prevista |
| 7. Presença, certificação e integração com IA | 24/11 a 07/12/2026 | Check-in, certificados em PDF e extração automática de dados | Prevista |
| 8. Testes, ajustes finais e apresentação | 08/12 a 15/12/2026 | Bateria de testes aprovada e sistema apresentado | Prevista |

A ordem das etapas obedece à dependência técnica entre elas. A etapa 6 precede obrigatoriamente a etapa 7, uma vez que o registro de presença e a emissão de certificados dependem da existência de inscrições confirmadas. De igual modo, a etapa 5 antecede toda a implementação, por estabelecer o ambiente no qual o código será verificado automaticamente desde a primeira contribuição.

Caso o prazo se mostre insuficiente, a equipe definiu que as funcionalidades assistidas por inteligência artificial, previstas na etapa 7, são as primeiras candidatas à redução de escopo. O sistema permanece íntegro e demonstrável sem elas, graças ao fluxo manual estabelecido pelo requisito RNF-02, o que não ocorreria com a supressão de qualquer item da etapa 6.

### 3.4.2 Métricas de Acompanhamento

O acompanhamento do projeto utiliza indicadores objetivos, aferidos a partir de dados já produzidos pelo próprio processo de desenvolvimento, sem exigir registro manual adicional por parte da equipe.

| Indicador | Forma de aferição | Meta |
|---|---|---|
| Requisitos funcionais implementados | Razão entre RF concluídos e os dez previstos | 100% ao término da etapa 7 |
| Aderência ao cronograma | Diferença entre a data prevista e a data real de conclusão de cada etapa | Desvio máximo de cinco dias por etapa |
| Cobertura de regras de negócio por testes | Razão entre as regras RN com teste automatizado e as onze previstas | Mínimo de 80% |
| Aprovação na integração contínua | Razão entre execuções aprovadas e execuções totais do pipeline | Mínimo de 90% |
| Revisão de código | Percentual de Pull Requests integrados com ao menos uma aprovação | 100% |

| Indicador | Forma de aferição | Meta |
|---|---|---|
| Distribuição de contribuições | Número de Pull Requests por integrante da equipe | Participação de todos os integrantes em cada etapa |

A escolha desses indicadores privilegia aquilo que pode ser verificado de forma automática no repositório e no pipeline de integração contínua. Métricas dependentes de estimativa subjetiva, como pontos por história ou velocidade de equipe, foram deliberadamente descartadas por introduzirem imprecisão sem benefício proporcional em um projeto desta dimensão.

## 3.5 Arquitetura do Sistema

A arquitetura descrita na seção 2 é detalhada, nesta especificação, por diagramas da Linguagem de Modelagem Unificada, um para cada perspectiva da solução.

A modelagem do sistema foi documentada por meio de seis diagramas da Linguagem de Modelagem Unificada, apresentados nas seções 3.1 e 3.7 a 3.11. Cada um deles aborda uma perspectiva distinta da solução, e em conjunto cobrem tanto a estrutura estática quanto o comportamento dinâmico da aplicação.

| Diagrama | Perspectiva | O que documenta |
|---|---|---|
| Casos de Uso | Funcional | Atores, fronteira do sistema e funcionalidades, com as relações de inclusão e extensão entre elas |
| Classes | Estrutural | Entidades do domínio com atributos e operações, associações e multiplicidades |
| Interação (sequência) | Comportamental | Troca de mensagens entre objetos no cadastro por IA e na emissão de certificado, com as barras de ativação |
| Atividades | Comportamental | Fluxo do cadastro assistido por IA, em raias de responsabilidade, com o desvio de contingência |
| Estados | Comportamental | Ciclo de vida da entidade Inscricao, da solicitação de vaga à emissão do certificado |
| Componentes | Arquitetural | Módulos internos do monolito e suas dependências com os serviços externos |

## 3.6 Modelo do Domínio

O modelo de domínio do Gerenciador de Eventos Acadêmicos foi estruturado para suportar o ciclo de vida completo dos eventos, garantindo a integridade relacional, o controle de acesso por papéis e o rastreamento das interações assistidas por inteligência artificial. A entidade Usuario armazena os dados fundamentais de acesso e identificação dos congressistas, incluindo nome, e-mail e a senha devidamente criptografada por meio de algoritmo de hash. Associada a ela, a entidade UsuarioPapel mapeia os perfis operacionais que o usuário pode acumular no sistema, participante ou organizador, definindo o nível de autorização para a execução das funcionalidades do software.

A gestão central das atividades acadêmicas é representada pela entidade Evento, que reúne informações essenciais como título, descrição, local, período de realização, carga horária, limite de vagas e situação atual, esta última controlada pelo enumerado StatusEvento. Cada evento é vinculado obrigatoriamente a um organizador, garantindo que apenas o seu criador

possua permissão para gerenciar suas configurações (RNF-04). As imagens de divulgação atreladas a cada atividade são gerenciadas pela entidade EventoFoto.

A relação entre os participantes e as atividades ocorre por meio da entidade Inscricao, que funciona como elemento central do modelo. É ela que controla o fluxo de acesso, a alocação em vagas garantidas, a ordenação na fila de espera e eventuais solicitações de cancelamento, alternando esses estados a partir do enumerado StatusInscricao, cujos valores persistidos são CONFIRMADA, EM_ESPERA, CANCELADA e AUSENTE.

A etapa de validação da participação e o fechamento do evento contam com as entidades Presenca, Certificado, Avaliacao e ResumoAvaliacao, todas vinculadas diretamente à Inscricao ou ao Evento. A entidade Presenca registra o check-in efetuado pelo organizador durante a realização do evento. Com base nesse registro, a entidade Certificado viabiliza a emissão do documento digital, atrelando a ele um código único de autenticidade para fins de verificação pública. Essa dependência torna explícita, no próprio modelo, a regra de que apenas participantes com presença registrada podem receber certificado.

Após o encerramento das atividades, os participantes enviam notas e pareceres registrados na entidade Avaliacao, cujos dados são posteriormente processados pela inteligência artificial para sintetizar os feedbacks consolidados na entidade ResumoAvaliacao. Por fim, a persistência do fluxo de cadastro inteligente é sustentada pela entidade InteracaoIA, que armazena o texto em linguagem natural submetido pelo usuário, o JSON estruturado devolvido via Function Calling e a data da interação, mantendo o histórico necessário para auditoria e para a reconstituição do cadastro em caso de falha.

## 3.7 Diagramas de Interação

Os dois diagramas de interação empregam barras de ativação sobre as linhas de vida, delimitando o período exato em que cada objeto permanece em execução. No diagrama de emissão de certificado, por exemplo, a ativação do serviço responsável estende-se da terceira à décima primeira mensagem, abrangendo a chamada à biblioteca de geração de PDF e evidenciando que nenhum arquivo é produzido antes da verificação do registro de presença.

**Figura 7 — Diagrama de Interação: Cadastro de Evento via IA.** Interpretação do texto do evento por saída estruturada. O fragmento loop representa a rotação entre modelos de linguagem até que um responda; o fragmento alt separa o retorno do rascunho do caminho de contingência, em que a interface recebe o código IA_INDISPONIVEL (RN-10). Toda chamada é registrada em interacao_ia, com ou sem sucesso.

**Figura 8 — Diagrama de Interação: Emissão de Certificado.** Emissão do certificado (RF-09). A operação é idempotente: um certificado já emitido é devolvido sem nova gravação. Sem registro de presença, a emissão é recusada com PRESENCA_NAO_REGISTRADA (RN-05). O código de autenticidade é gerado no construtor de Certificado, e nenhum arquivo PDF é produzido nesta etapa.

**Figura 9 — Diagrama de Interação: Download do Certificado.** Obtenção do certificado em PDF, em requisição separada da emissão. O documento é montado pela biblioteca iText a cada solicitação, em memória e sem gravação em disco, e somente o titular da inscrição pode obtê-lo (RN-08).

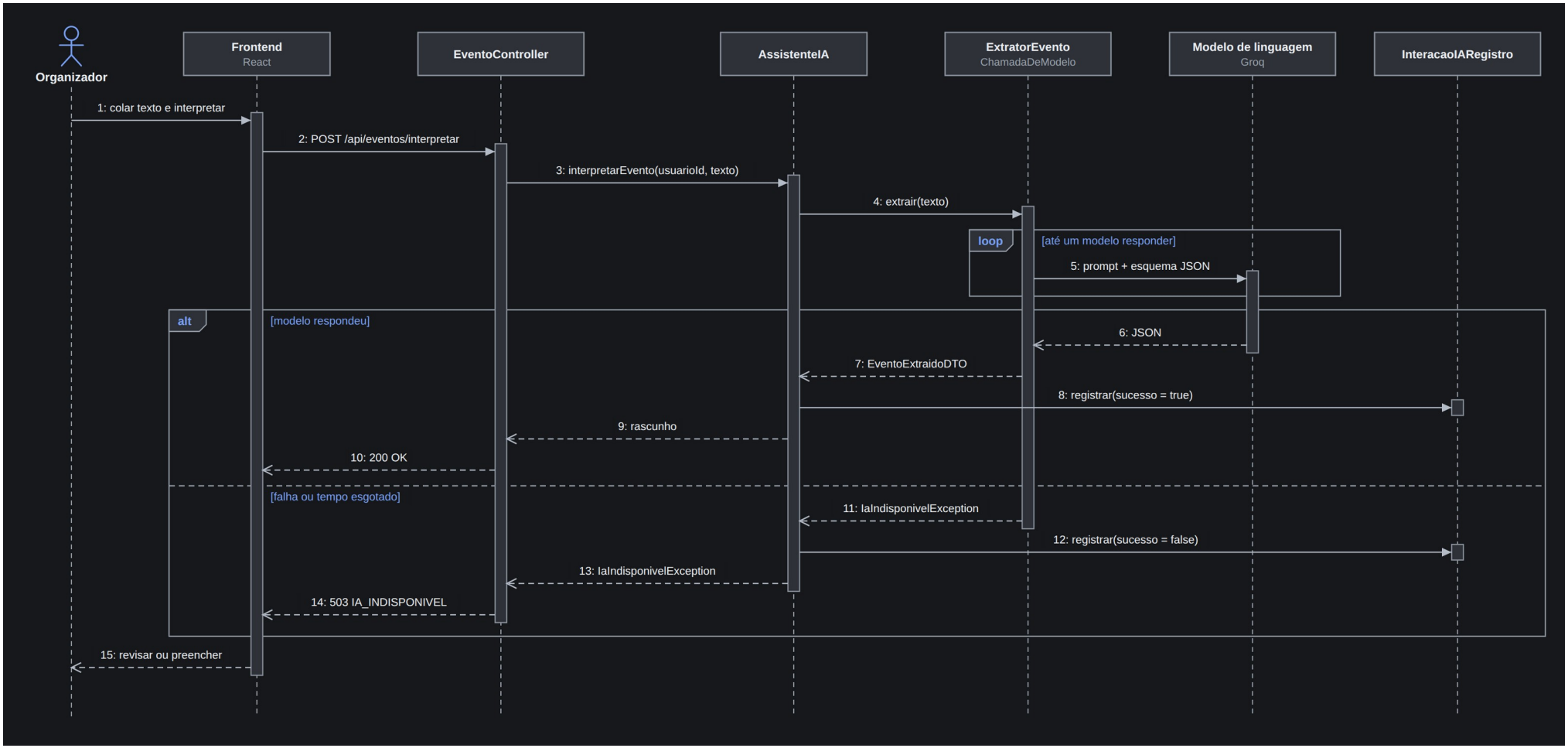



**Figura 7 — Diagrama de Interação: Cadastro de Evento via IA**

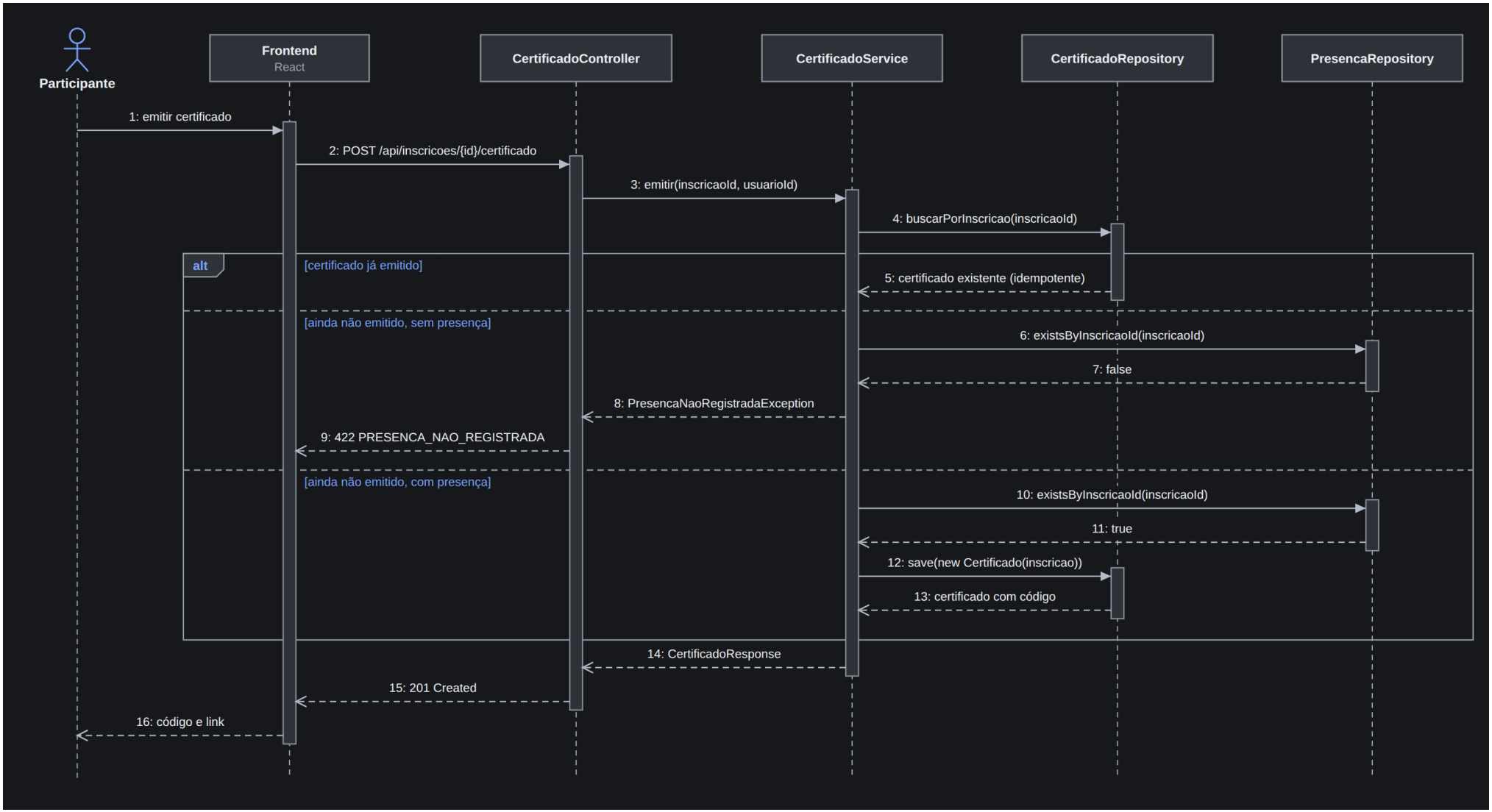



**Figura 8 — Diagrama de Interação: Emissão de Certificado**

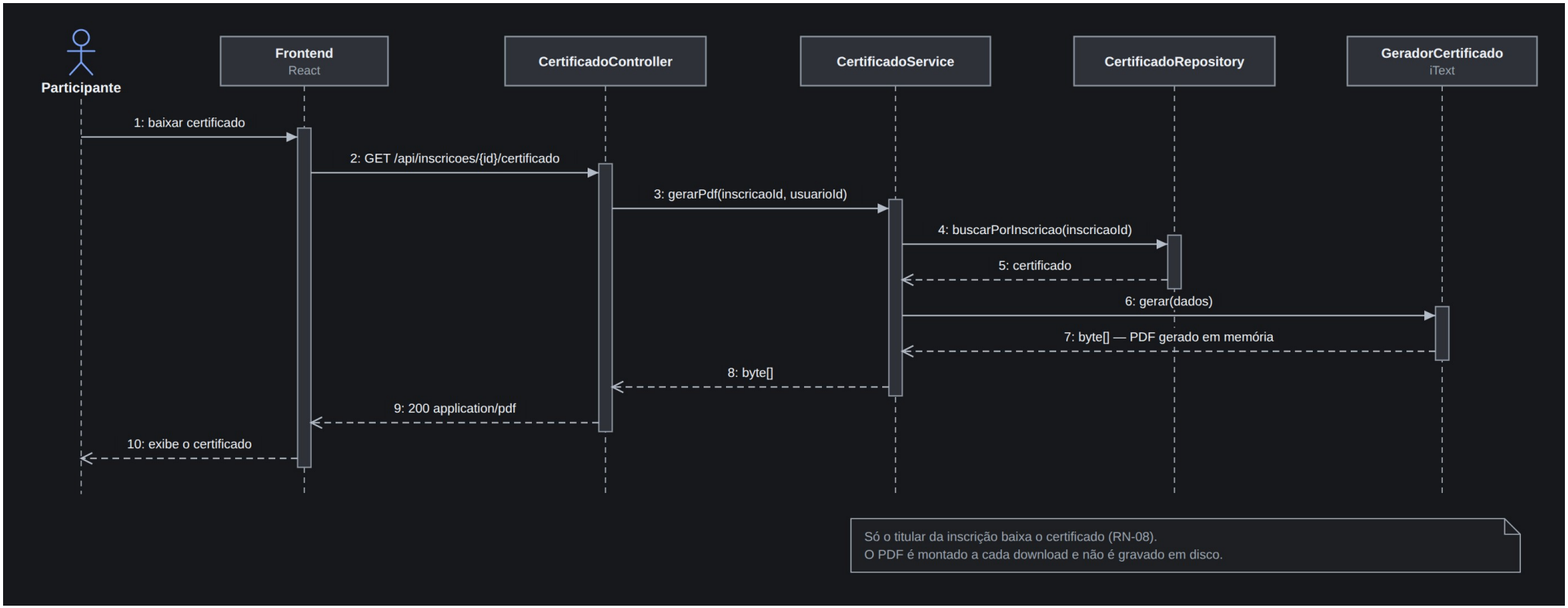



**Figura 9 — Diagrama de Interação: Download do Certificado**

## 3.8 Diagrama de Classes

**Figura 10 — Diagrama de Classes: Usuários, Eventos e Inscrições.** Núcleo do domínio, com os atributos e as operações públicas existentes no código; operações sublinhadas são estáticas. Inscricao liga Usuario a Evento e concentra a situação da vaga e a posição na fila (RN-01 a RN-03). O atributo versao de Evento sustenta o controle de concorrência na disputa pela última vaga.

**Figura 11 — Diagrama de Classes: Participação e Inteligência Artificial.** Registros que dependem da inscrição — presença, certificado e avaliação, no máximo um de cada por inscrição (RN-05 e RN-06) — e as entidades de IA: o resumo consolidado por evento (RN-07) e o histórico de interações com o modelo, mantido para auditoria (RN-10). Inscricao, Evento e Usuario aparecem apenas como referência à Figura 10.

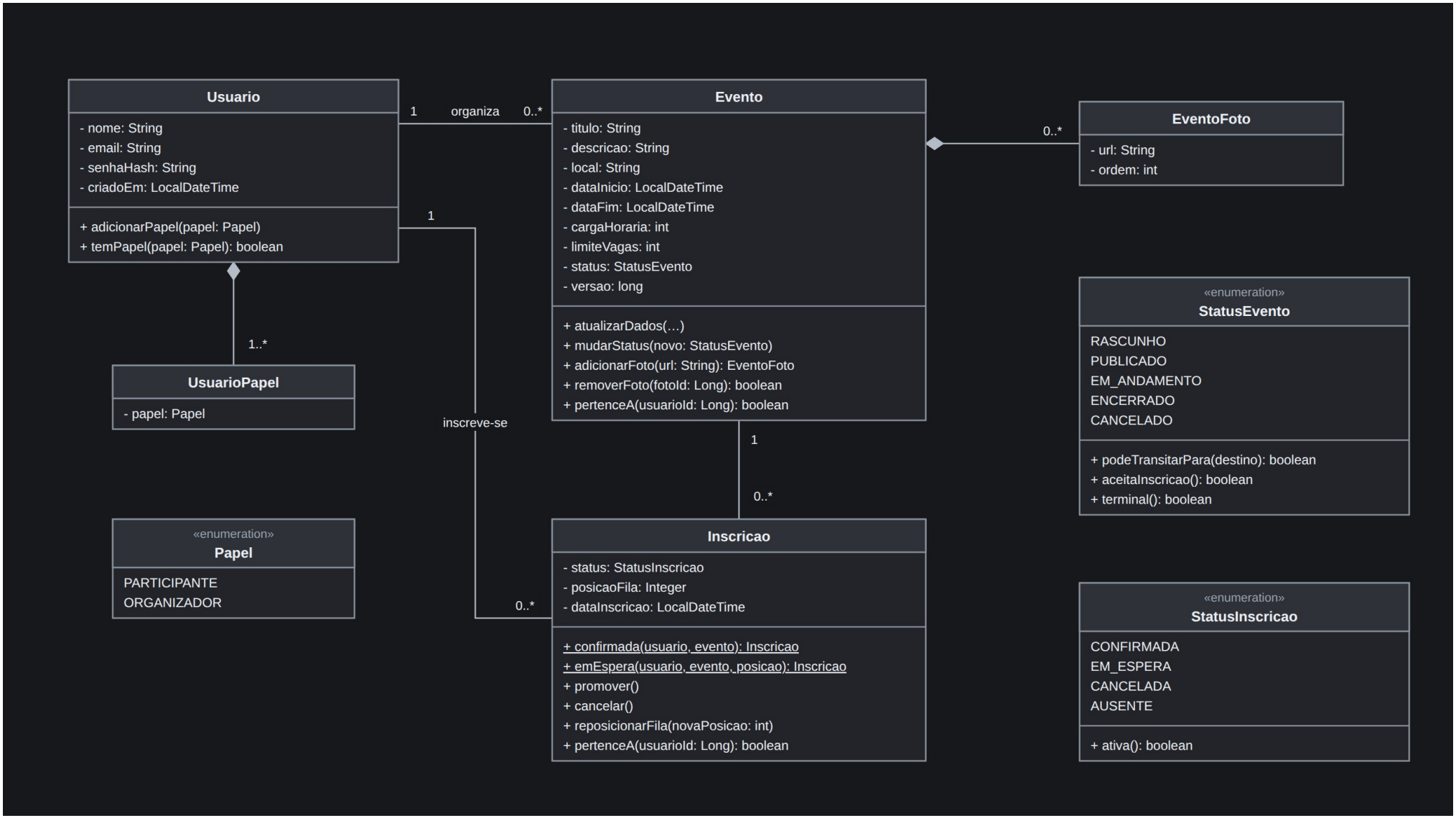



**Figura 10 — Diagrama de Classes: Usuários, Eventos e Inscrições**

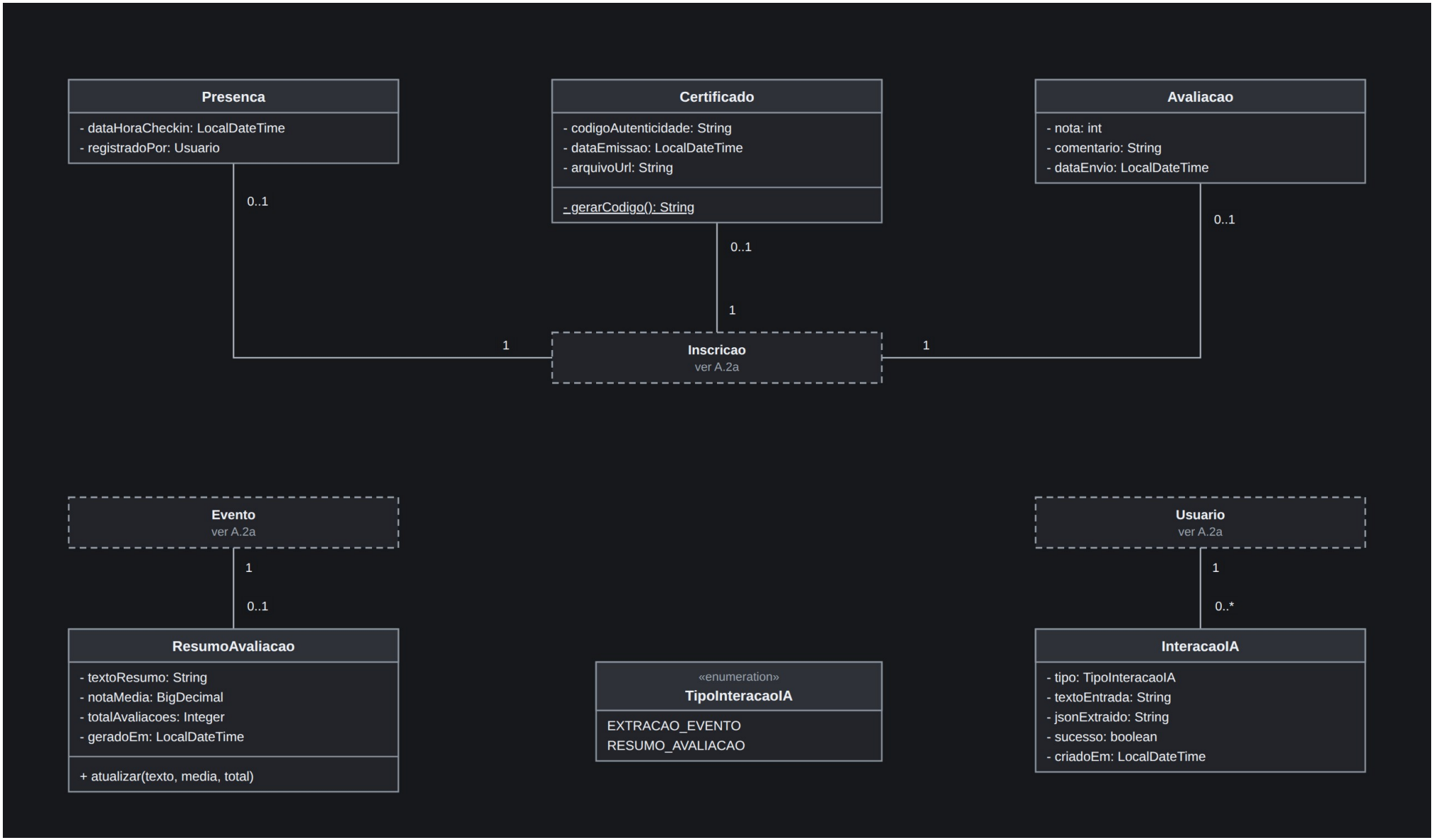



**Figura 11 — Diagrama de Classes: Participação e Inteligência Artificial**

## 3.9 Diagrama de Atividades

**Figura 12 — Diagrama de Atividades: Cadastro de Evento Assistido por IA.** Fluxo do cadastro a partir de texto livre (RF-03), em raias de responsabilidade. Uma resposta inválida — falha do serviço, rotação de modelos esgotada ou tempo excedido — conduz ao preenchimento manual (RN-10). Em ambos os caminhos, o evento só é gravado após a revisão do organizador, em situação de rascunho.

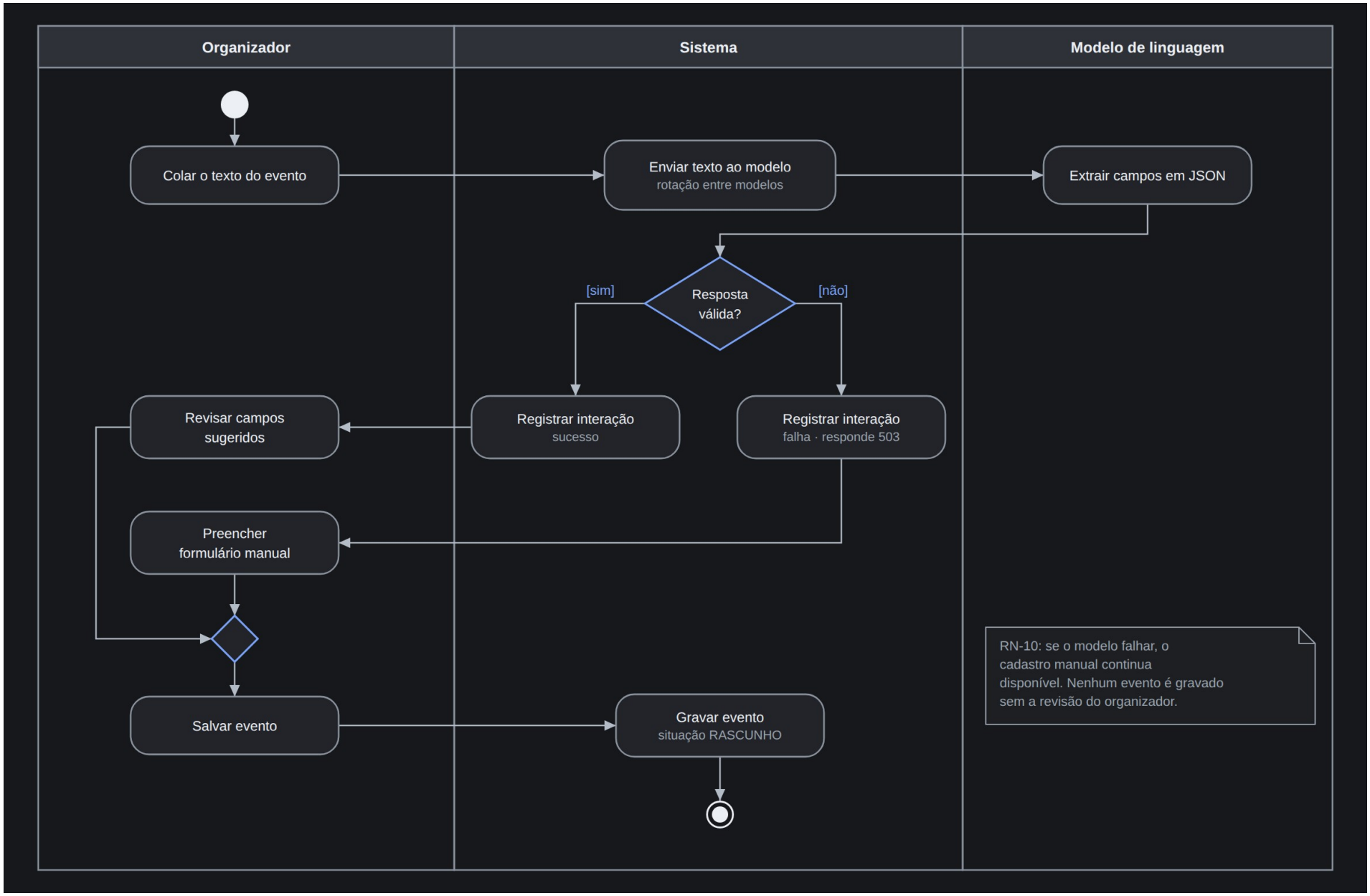



**Figura 12 — Diagrama de Atividades: Cadastro de Evento Assistido por IA**

### 3.10 Diagrama de Estados

**Figura 13 — Diagrama de Estados: Inscrição.** Ciclo de vida persistido da inscrição, cobrindo a garantia de vaga (RF-05), a fila de espera e a promoção por desistência (RF-06) e o cancelamento (RF-07). Presença e certificado não são estados, e sim registros vinculados a uma inscrição confirmada. O valor AUSENTE existe na enumeração, mas nenhum fluxo da versão atual o grava.

**Figura 14 — Diagrama de Estados: Evento.** Ciclo de vida do evento conforme as transições permitidas por StatusEvento. Somente a situação PUBLICADO aceita inscrições, e as situações ENCERRADO e CANCELADO são terminais, sem edição nem novas inscrições (RN-09).

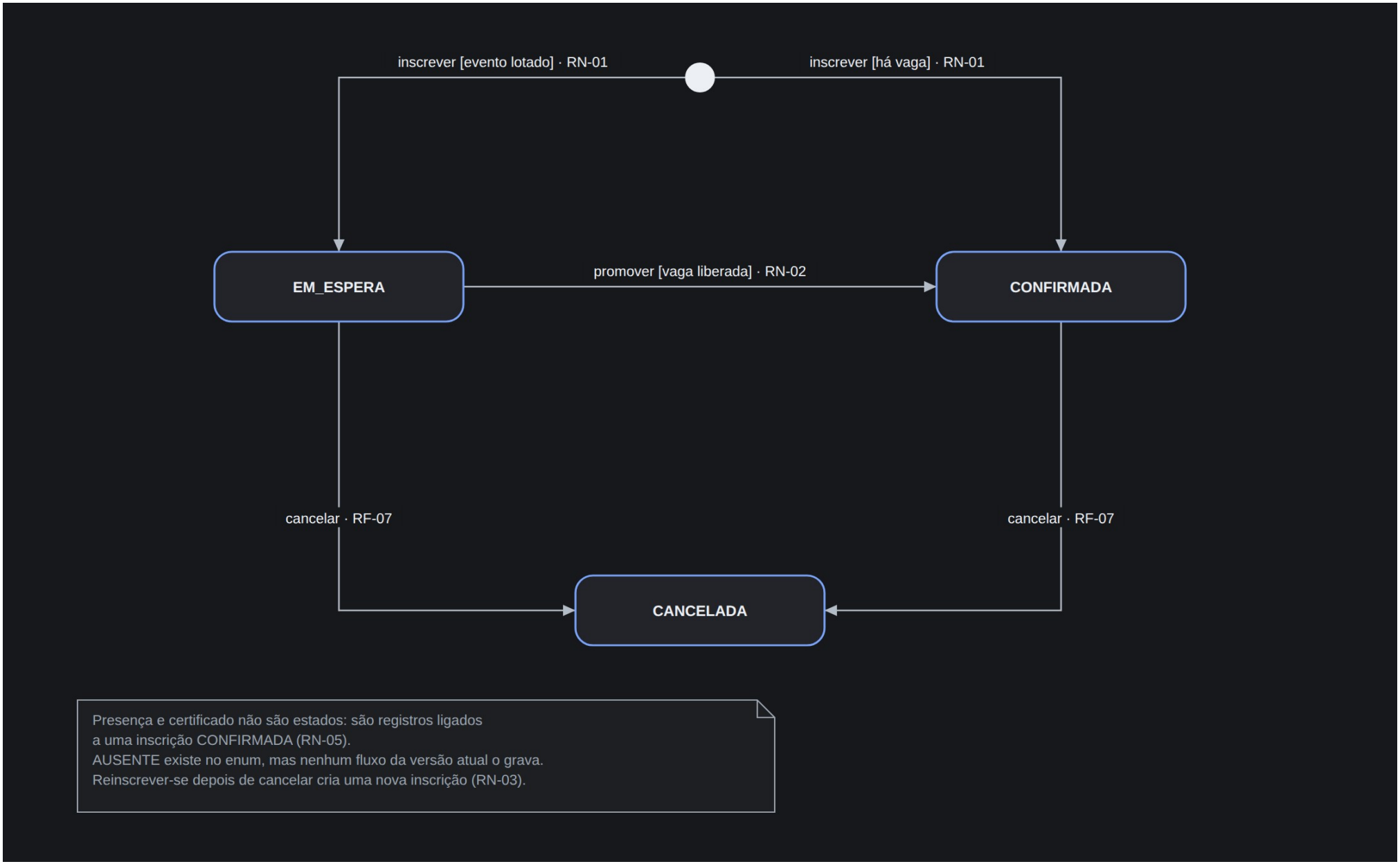



**Figura 13 — Diagrama de Estados: Inscrição**

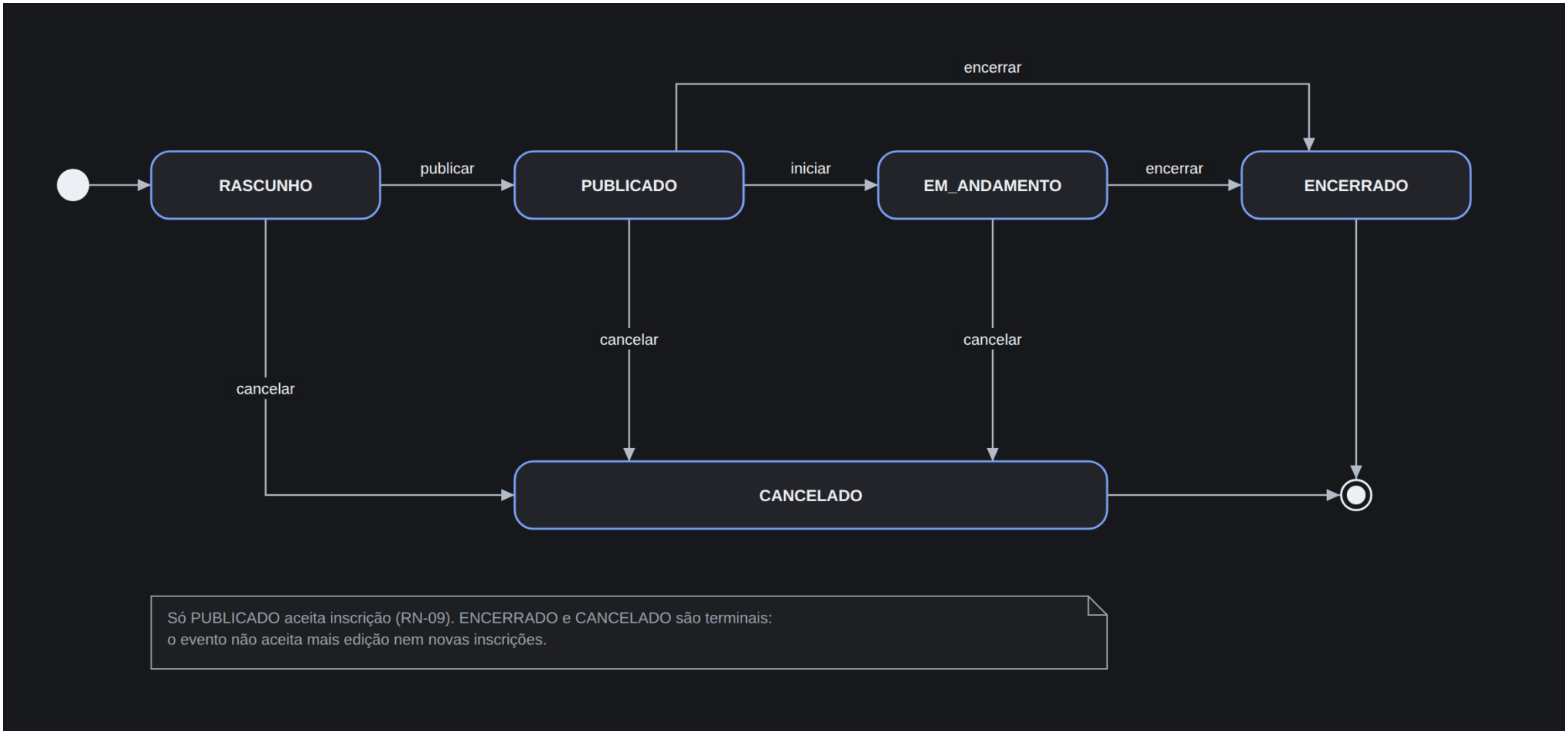



**Figura 14 — Diagrama de Estados: Evento**

### 3.11 Diagrama de Componentes

**Figura 15 — Diagrama de Componentes.** Arquitetura do monólito em camadas. Toda requisição da interface passa pelo filtro de segurança antes de alcançar os módulos de domínio. As dependências tracejadas para o módulo de IA são opcionais (RN-10), e a chave do serviço de linguagem permanece restrita ao servidor (RNF-03).

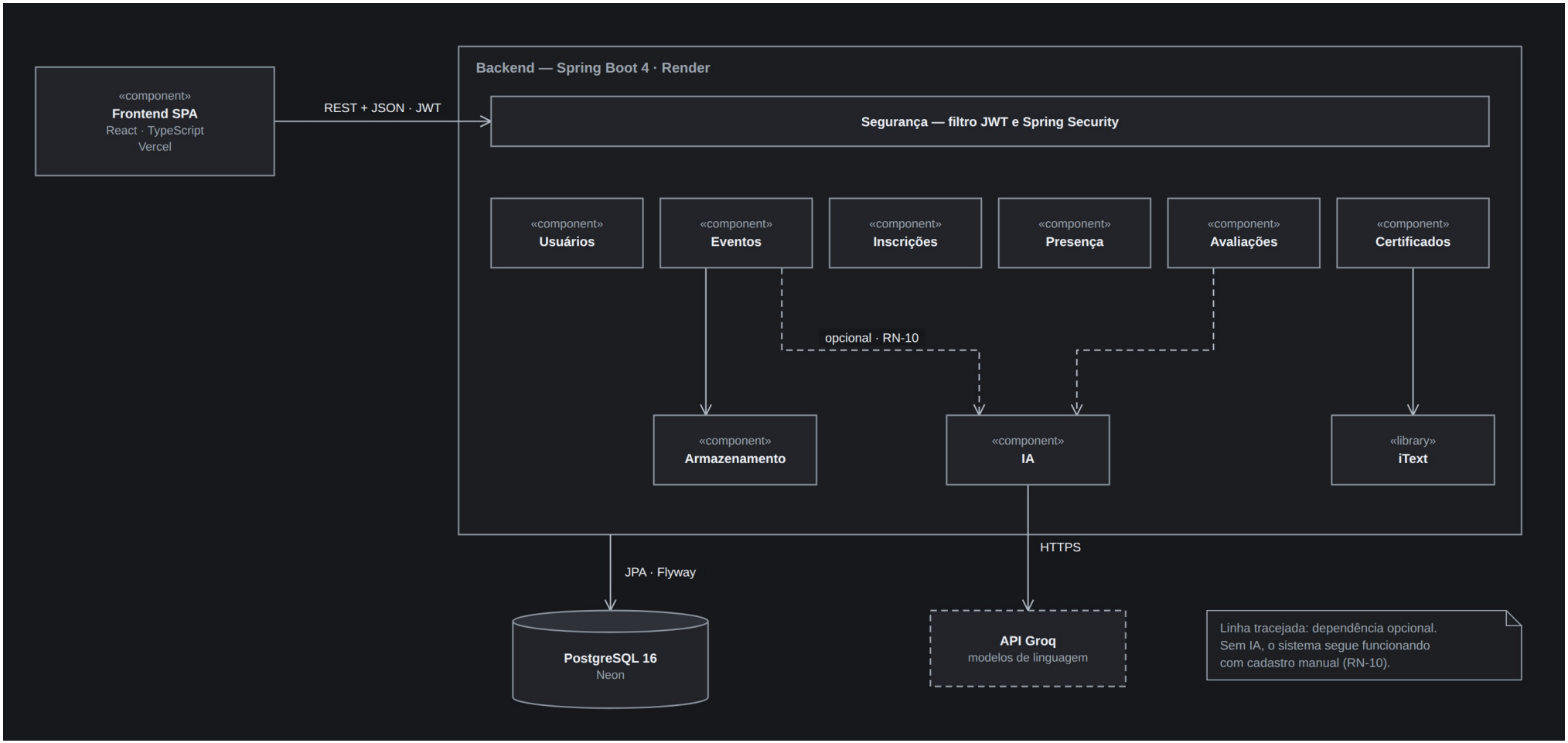



**Figura 15 — Diagrama de Componentes**

## 3.12 Modelo de Dados

O modelo de dados é apresentado em três níveis complementares: o modelo lógico, que traduz as entidades do domínio em relações; a criação física, que materializa essas relações no PostgreSQL por meio de migrações versionadas; e o dicionário de dados, que detalha cada campo.

## 3.13 Modelo Lógico da Base de Dados

O modelo lógico traduz as entidades descritas na seção 3.6 em relações do modelo relacional, com a definição das chaves primárias, das chaves estrangeiras e das cardinalidades que governam os vínculos entre as tabelas. Todas as relações utilizam chave primária artificial numérica, de geração automática, estratégia adotada para simplificar os relacionamentos e evitar chaves compostas.

O relacionamento entre usuários e eventos é de muitos para muitos e foi resolvido por meio da relação INSCRICAO, que deixa de ser uma simples tabela associativa e passa a concentrar atributos próprios, como situação e posição na fila de espera. As relações PRESENCA, CERTIFICADO e AVALIACAO mantêm vínculo de um para um com INSCRICAO, de modo que a dependência funcional entre presença e emissão de certificado fique assegurada pela própria estrutura do banco.

| Relação | Chave primária | Chaves estrangeiras | Cardinalidade |
|---|---|---|---|
| USUARIO | id | — | 1:N com USUARIO_PAPEL, EVENTO, INSCRICAO |
| USUARIO_PAPEL | id | usuario_id → USUARIO | N:1 com USUARIO |
| EVENTO | id | organizador_id → USUARIO | 1:N com EVENTO_FOTO e INSCRICAO |
| EVENTO_FOTO | id | evento_id → EVENTO | N:1 com EVENTO |
| INSCRICAO | id | usuario_id → USUARIO; evento_id → EVENTO | N:1 com USUARIO e EVENTO |
| PRESENCA | id | inscricao_id → INSCRICAO; registrado_por → USUARIO | 1:1 com INSCRICAO |
| CERTIFICADO | id | inscricao_id → INSCRICAO | 1:1 com INSCRICAO |
| AVALIACAO | id | inscricao_id → INSCRICAO | 1:1 com INSCRICAO |
| RESUMO_AVALIACAO | id | evento_id → EVENTO | 1:1 com EVENTO |
| INTERACAO_IA | id | usuario_id → USUARIO | N:1 com USUARIO |

Duas restrições de unicidade merecem destaque no modelo. A primeira impede o cadastro de dois usuários com o mesmo endereço de correio eletrônico. A segunda é uma restrição parcial sobre INSCRICAO, que proíbe a existência de duas inscrições ativas do mesmo usuário em um mesmo evento, mas permite a reinscrição posterior ao cancelamento, atendendo à regra RN-03.

A normalização foi conduzida até a terceira forma normal. Valores como a quantidade de vagas ocupadas e a média das avaliações não são armazenados de forma redundante: o primeiro é obtido por contagem das inscrições confirmadas, e o segundo é persistido apenas na

relação RESUMO_AVALIACAO, por constituir resultado consolidado de um processamento posterior ao encerramento do evento.

### 3.14 Criação Física do Modelo de Dados

A criação física da base de dados é realizada por meio de scripts versionados em conjunto com o código-fonte, executados automaticamente pela ferramenta de migração Flyway durante a inicialização da aplicação. Essa abordagem garante que todos os ambientes partam de uma estrutura idêntica e que qualquer alteração de esquema fique registrada no histórico do repositório, podendo ser auditada e reproduzida.

Os scripts obedecem à nomenclatura sequencial exigida pela ferramenta, iniciando pelo arquivo V1__schema.sql, responsável pela criação integral das tabelas, restrições e índices descritos a seguir. Alterações posteriores não modificam esse arquivo, sendo registradas em novos scripts numerados. O sistema gerenciador de banco de dados adotado é o PostgreSQL.

```
CREATE TABLE usuario (
    id          BIGSERIAL    PRIMARY KEY,
    nome        VARCHAR(120) NOT NULL,
    email       VARCHAR(160) NOT NULL UNIQUE,
    senha_hash  VARCHAR(100) NOT NULL,
    criado_em   TIMESTAMP    NOT NULL DEFAULT now()
);

CREATE TABLE usuario_papel (
    id          BIGSERIAL   PRIMARY KEY,
    usuario_id  BIGINT      NOT NULL REFERENCES usuario(id) ON DELETE CASCADE,
    papel       VARCHAR(20) NOT NULL,
    UNIQUE (usuario_id, papel)
);

CREATE TABLE evento (
    id              BIGSERIAL    PRIMARY KEY,
    organizador_id  BIGINT       NOT NULL REFERENCES usuario(id),
    titulo          VARCHAR(160) NOT NULL,
    descricao       TEXT         NOT NULL,
    local           VARCHAR(200) NOT NULL,
    data_inicio     TIMESTAMP    NOT NULL,
    data_fim        TIMESTAMP    NOT NULL,
    carga_horaria   INT          NOT NULL CHECK (carga_horaria > 0),
    limite_vagas    INT          NOT NULL CHECK (limite_vagas > 0),
    status          VARCHAR(20)  NOT NULL DEFAULT 'RASCUNHO',
    criado_em       TIMESTAMP    NOT NULL DEFAULT now(),
    versao          BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT ck_evento_periodo CHECK (data_fim > data_inicio)
);
CREATE INDEX idx_evento_status_inicio ON evento (status, data_inicio);

CREATE TABLE evento_foto (
    id         BIGSERIAL    PRIMARY KEY,
    evento_id  BIGINT       NOT NULL REFERENCES evento(id) ON DELETE CASCADE,
    url        VARCHAR(500) NOT NULL,
    ordem      INT          NOT NULL DEFAULT 0
);

CREATE TABLE inscricao (
    id              BIGSERIAL   PRIMARY KEY,
    usuario_id      BIGINT      NOT NULL REFERENCES usuario(id),
    evento_id       BIGINT      NOT NULL REFERENCES evento(id),
    status          VARCHAR(20) NOT NULL,
    posicao_fila    INT,
```

```
    data_inscricao  TIMESTAMP   NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX ux_inscricao_ativa
    ON inscricao (usuario_id, evento_id)
    WHERE status <> 'CANCELADA';
CREATE INDEX idx_inscricao_fila ON inscricao (evento_id, status, posicao_fila);

CREATE TABLE presenca (
    id                BIGSERIAL PRIMARY KEY,
    inscricao_id      BIGINT    NOT NULL UNIQUE REFERENCES inscricao(id),
    data_hora_checkin TIMESTAMP NOT NULL DEFAULT now(),
    registrado_por    BIGINT    NOT NULL REFERENCES usuario(id)
);

CREATE TABLE certificado (
    id                    BIGSERIAL    PRIMARY KEY,
    inscricao_id          BIGINT       NOT NULL UNIQUE REFERENCES inscricao(id),
    codigo_autenticidade  VARCHAR(40)  NOT NULL UNIQUE,
    data_emissao          TIMESTAMP    NOT NULL DEFAULT now(),
    arquivo_url           VARCHAR(500)
);

CREATE TABLE avaliacao (
    id            BIGSERIAL PRIMARY KEY,
    inscricao_id  BIGINT    NOT NULL UNIQUE REFERENCES inscricao(id),
    nota          INT       NOT NULL CHECK (nota BETWEEN 1 AND 5),
    comentario    TEXT,
    data_envio    TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE resumo_avaliacao (
    id                BIGSERIAL    PRIMARY KEY,
    evento_id         BIGINT       NOT NULL UNIQUE REFERENCES evento(id),
    texto_resumo      TEXT         NOT NULL,
    nota_media        NUMERIC(3,2),
    total_avaliacoes  INT,
    gerado_em         TIMESTAMP    NOT NULL DEFAULT now()
);

CREATE TABLE interacao_ia (
    id             BIGSERIAL   PRIMARY KEY,
    usuario_id     BIGINT      NOT NULL REFERENCES usuario(id),
    tipo           VARCHAR(30) NOT NULL,
    texto_entrada  TEXT        NOT NULL,
    json_extraido  TEXT,
    sucesso        BOOLEAN     NOT NULL,
    criado_em      TIMESTAMP   NOT NULL DEFAULT now()
);
```

Os índices criados atendem aos padrões de consulta previstos. O índice sobre situação e data de início do evento sustenta a listagem pública exigida pelo requisito RNF-01, enquanto o índice sobre evento, situação e posição na fila acelera a localização do próximo participante a ser promovido em caso de desistência.

### 3.15 Dicionário de Dados

O dicionário de dados descreve cada campo das tabelas do sistema, informando o tipo de dado adotado, as restrições aplicadas e o significado da informação armazenada. Os campos identificadores seguem o mesmo padrão em todas as tabelas: numéricos, de geração automática pelo banco e sem significado de negócio.

### 3.15.1 Tabela USUARIO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador do usuário |
| nome | VARCHAR(120) | NOT NULL | Nome completo do usuário |
| email | VARCHAR(160) | NOT NULL, UNIQUE | Endereço de correio eletrônico, utilizado como credencial de acesso |
| senha_hash | VARCHAR(100) | NOT NULL | Resumo criptográfico da senha (RN-11) |
| criado_em | TIMESTAMP | NOT NULL | Data e hora do cadastro |

### 3.15.2 Tabela USUARIO_PAPEL

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador do vínculo |
| usuario_id | BIGINT | FK, NOT NULL | Usuário ao qual o papel pertence |
| papel | VARCHAR(20) | NOT NULL | Perfil atribuído: PARTICIPANTE ou ORGANIZADOR |

### 3.15.3 Tabela EVENTO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador do evento |
| organizador_id | BIGINT | FK, NOT NULL | Usuário responsável pelo evento (RN-08) |
| titulo | VARCHAR(160) | NOT NULL | Título de divulgação |
| descricao | TEXT | NOT NULL | Descrição detalhada da atividade |
| local | VARCHAR(200) | NOT NULL | Local de realização |
| data_inicio | TIMESTAMP | NOT NULL | Início da realização |
| data_fim | TIMESTAMP | NOT NULL, > data_inicio | Encerramento da realização |
| carga_horaria | INT | NOT NULL, > 0 | Carga horária registrada no certificado |
| limite_vagas | INT | NOT NULL, > 0 | Número máximo de inscrições confirmadas (RN-01) |
| status | VARCHAR(20) | NOT NULL | RASCUNHO, PUBLICADO, EM_ANDAMENTO, ENCERRADO ou CANCELADO |
| criado_em | TIMESTAMP | NOT NULL | Data e hora da criação |
| versao | BIGINT | NOT NULL | Controle de concorrência no acesso simultâneo às vagas |

### 3.15.4 Tabela EVENTO_FOTO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador da imagem |
| evento_id | BIGINT | FK, NOT NULL | Evento ao qual a imagem pertence |

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| url | VARCHAR(500) | NOT NULL | Endereço de acesso ao arquivo |
| ordem | INT | NOT NULL | Posição de exibição na divulgação |

### 3.15.5 Tabela INSCRICAO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador da inscrição |
| usuario_id | BIGINT | FK, NOT NULL | Participante inscrito |
| evento_id | BIGINT | FK, NOT NULL | Evento da inscrição |
| status | VARCHAR(20) | NOT NULL | CONFIRMADA, EM_ESPERA, CANCELADA ou AUSENTE |
| posicao_fila | INT | Nulo quando confirmada | Posição na fila de espera (RN-01 e RN-02) |
| data_inscricao | TIMESTAMP | NOT NULL | Data e hora do pedido de inscrição |

### 3.15.6 Tabela PRESENCA

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador do registro de presença |
| inscricao_id | BIGINT | FK, NOT NULL, UNIQUE | Inscrição correspondente |
| data_hora_checkin | TIMESTAMP | NOT NULL | Momento do registro de presença (RN-04) |
| registrado_por | BIGINT | FK, NOT NULL | Organizador que efetuou o registro |

### 3.15.7 Tabela CERTIFICADO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador do certificado |
| inscricao_id | BIGINT | FK, NOT NULL, UNIQUE | Inscrição que originou a emissão (RN-05) |
| codigo_autenticidade | VARCHAR(40) | NOT NULL, UNIQUE | Código utilizado na validação pública |
| data_emissao | TIMESTAMP | NOT NULL | Data e hora da emissão |
| arquivo_url | VARCHAR(500) | Opcional | Endereço do arquivo, quando houver armazenamento |

### 3.15.8 Tabela AVALIACAO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador da avaliação |
| inscricao_id | BIGINT | FK, NOT NULL, UNIQUE | Inscrição do avaliador (RN-06) |
| nota | INT | NOT NULL, de 1 a 5 | Nota atribuída ao evento |
| comentario | TEXT | Opcional | Parecer descritivo do participante |
| data_envio | TIMESTAMP | NOT NULL | Data e hora do envio |

### 3.15.9 Tabela RESUMO_AVALIACAO

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador do resumo |
| evento_id | BIGINT | FK, NOT NULL, UNIQUE | Evento resumido |
| texto_resumo | TEXT | NOT NULL | Síntese gerada por inteligência artificial (RN-07) |
| nota_media | NUMERIC(3,2) | Opcional | Média das notas recebidas |
| total_avaliacoes | INT | Opcional | Quantidade de avaliações consideradas |
| gerado_em | TIMESTAMP | NOT NULL | Data e hora da geração |

### 3.15.10 Tabela INTERACAO_IA

| Campo | Tipo | Restrições | Descrição |
|---|---|---|---|
| id | BIGSERIAL | PK | Identificador da interação |
| usuario_id | BIGINT | FK, NOT NULL | Usuário que originou a solicitação |
| tipo | VARCHAR(30) | NOT NULL | EXTRACAO_EVENTO ou RESUMO_AVALIACAO |
| texto_entrada | TEXT | NOT NULL | Texto em linguagem natural submetido ao modelo |
| json_extraido | TEXT | Opcional | Retorno estruturado devolvido pelo modelo |
| sucesso | BOOLEAN | NOT NULL | Indica se a chamada foi concluída com êxito (RN-10) |
| criado_em | TIMESTAMP | NOT NULL | Data e hora da interação |

## 3.16 Ambiente de Desenvolvimento

O ambiente de desenvolvimento foi padronizado de modo que qualquer integrante da equipe consiga executar o sistema completo em sua própria máquina, sem depender de serviços remotos. Essa padronização reduz divergências de comportamento entre as estações de trabalho e assegura que o código verificado localmente se comporte da mesma maneira no ambiente de integração contínua.

| Componente | Versão de referência | Finalidade |
|---|---|---|
| JDK | Java 21 (LTS) | Compilação e execução do servidor de aplicação |
| Maven | Wrapper incluído no repositório | Gerenciamento de dependências e automação da construção |
| PostgreSQL | 16 ou superior (em produção, serviço gerenciado na Neon) | Banco de dados relacional, executado em contêiner |
| Docker e Docker Compose | Versão corrente | Provisionamento do banco de dados local |
| Node.js | 22 (versão utilizada na integração contínua) | Construção e execução da interface web |
| Git | Versão corrente | Controle de versão e integração com o repositório remoto |

| Componente | Versão de referência | Finalidade |
|---|---|---|
| IDE | IntelliJ IDEA ou Visual Studio Code | Edição de código e depuração |
| Ollama | Opcional | Execução local de modelo de linguagem durante o desenvolvimento |

A execução local do modelo de linguagem por meio do Ollama é opcional e exige espaço em disco e memória consideráveis. Durante o desenvolvimento deste projeto optou-se pelo serviço remoto Groq, cujo plano gratuito dispensa cartão de crédito e cuja cota é vinculada à conta, bastando alterar o perfil ativo da aplicação para trocar de provedor. A interface de programação do Groq é compatível com a da OpenAI, de modo que a mesma implementação atende aos quatro perfis previstos.

O projeto adota perfis de configuração distintos para desenvolvimento, teste e produção. O perfil de teste utiliza uma instância de banco provisionada automaticamente durante a execução da bateria de testes, de forma que a base de desenvolvimento nunca seja alterada por um teste automatizado. A configuração local reside em um arquivo .env na raiz do repositório, carregado na inicialização e mantido fora do controle de versão; essa escolha substituiu a passagem de parâmetros pela linha de comando, que apresenta incompatibilidade com o interpretador de comandos do Windows. Na ausência de perfil explícito, a aplicação assume o perfil de desenvolvimento.

### 3.16.1 Pré-requisitos

Para o desenvolvimento e o funcionamento do Gerenciador de Eventos Acadêmicos é necessário um ambiente compatível com as tecnologias utilizadas no projeto. O sistema utiliza Java 21 com Spring Boot para o desenvolvimento da aplicação, PostgreSQL para o armazenamento dos dados e Spring AI para a integração com os modelos de linguagem. Também é necessário configurar o serviço de inteligência artificial utilizado pela aplicação: o projeto adota o Groq, cuja chave é obtida sem cartão de crédito, admitindo igualmente o OpenRouter, a OpenAI ou uma solução local por meio do Ollama.

### 3.16.2 Passos de Instalação

A sequência abaixo descreve a instalação completa do ambiente local, partindo de uma máquina na qual estejam disponíveis os componentes relacionados na seção 3.16. Os comandos devem ser executados a partir da raiz do repositório.

a) Obter o código-fonte a partir do repositório remoto e acessar o diretório criado.

```
git clone <url-do-repositorio>
cd GerenciadorDeEventos
```

b) Criar o arquivo de variáveis de ambiente a partir do modelo versionado e preencher os valores locais, conforme a relação do Apêndice A. A aplicação lê esse arquivo na inicialização, o que dispensa a definição de variáveis na sessão do terminal.

```
cp .env.example .env
```

c) Provisionar o banco de dados local em contêiner e aguardar a inicialização.

```
docker compose up -d
```

d) Executar a aplicação. As migrações do Flyway são aplicadas automaticamente na inicialização, criando todas as tabelas descritas na seção 3.14. Em ambiente Windows, o executável é invocado como .\mvnw.cmd spring-boot:run.

```
./mvnw spring-boot:run
```

e) Instalar as dependências da interface web e iniciá-la em modo de desenvolvimento.

```
cd frontend
npm install
```

```
npm run dev
```

f) Verificar a instalação acessando o endereço de verificação de saúde do servidor, que deve responder indicando situação operacional.

```
curl http://localhost:8080/actuator/health
```

Caso a inicialização seja interrompida com erro relativo às migrações, a causa mais frequente é a existência de uma base anterior com estrutura divergente. O procedimento de correção está descrito na seção 16.

### 3.17 Sistemas e Componentes Externos Utilizados

A aplicação depende de um conjunto reduzido de serviços e bibliotecas de terceiros, relacionados a seguir com a respectiva finalidade e a consequência de sua eventual indisponibilidade. A coluna de criticidade indica se a falha do componente interrompe o funcionamento do sistema ou apenas degrada parte das funcionalidades.

| Componente | Tipo | Finalidade no sistema | Criticidade |
|---|---|---|---|
| PostgreSQL (Neon) | Serviço gerenciado | Persistência de todos os dados da aplicação | Crítica — indisponibiliza o sistema |
| Groq, OpenRouter, OpenAI ou Ollama | Serviço de linguagem | Extração de dados de texto livre e geração de resumos; em produção utiliza-se o Groq | Baixa — há fluxo manual (RNF-02) |
| Spring AI | Biblioteca | Intermediação das chamadas ao modelo de linguagem com saída estruturada | Baixa |
| iText | Biblioteca | Montagem dos certificados digitais em PDF | Média — impede a emissão de certificados |
| Flyway | Biblioteca | Versionamento e aplicação das migrações de banco | Crítica na inicialização |
| Render | Plataforma de hospedagem | Execução do servidor de aplicação em produção | Crítica em produção |
| Vercel | Plataforma de hospedagem | Publicação da interface web em produção | Crítica em produção |
| GitHub Actions | Integração contínua | Verificação automática de cada contribuição | Baixa — não afeta a execução |
| Testcontainers | Biblioteca de teste | Provisionamento de banco descartável durante a bateria de integração | Baixa — não afeta a execução |
| Playwright | Biblioteca de teste | Execução dos testes de ponta a ponta em navegador real | Baixa — não afeta a execução |

Cabe destacar que a biblioteca iText é distribuída sob licença AGPL, compatível com a natureza acadêmica e de código aberto deste projeto. Caso o sistema venha a ser utilizado em contexto comercial fechado, será necessário adquirir licença específica ou substituir a biblioteca, motivo pelo qual a geração de documentos foi isolada atrás de uma interface própria, de modo que a troca não afete as demais camadas da aplicação.

## 3.18 Tecnologia Emergente e Sua Utilização

A integração de tecnologias emergentes em ecossistemas de software modernos representa um avanço significativo na automação de processos e na melhoria da experiência do usuário, especialmente em plataformas de gestão. No contexto do Gerenciador de Eventos Acadêmicos, a adoção da Inteligência Artificial atua como elemento central para a modernização das interações, substituindo formulários estáticos convencionais por um módulo de atendimento e cadastro pautado em processamento de linguagem natural.

A aplicação prática dessa inovação baseia-se no uso do recurso de saída estruturada, intermediado pelo framework Spring AI, que conecta a aplicação Java Spring Boot a modelos de linguagem hospedados no Groq, no OpenRouter ou na OpenAI, bem como a soluções locais executadas via Ollama. O framework anexa à requisição o esquema do objeto esperado e converte a resposta em um registro Java, de modo que o sistema interprete textos brutos colados pelos organizadores, extraia os parâmetros necessários e os receba já estruturados. Os dados extraídos são então validados e persistidos no banco de dados relacional PostgreSQL. As instruções enviadas ao modelo proíbem explicitamente a inferência de valores ausentes: campos não mencionados no texto retornam vazios e são devolvidos ao organizador em uma relação própria, em vez de preenchidos por dedução.

Além de viabilizar a criação simplificada de eventos e a mediação de dúvidas com participantes, a inteligência artificial é empregada na análise de dados pós-evento para a geração automatizada de resumos executivos a partir do feedback recebido dos congressistas. Complementarmente, o ecossistema integra a biblioteca iText para a emissão automatizada de certificados digitais em formato PDF, com códigos de autenticidade atrelados à validação prévia de presença dos participantes.

Para assegurar a robustez do sistema frente a eventuais falhas nos serviços de linguagem, a arquitetura adota um princípio de resiliência em dois níveis. O primeiro é a rotação de modelos: a aplicação aceita uma lista de modelos e percorre-a até que um responda, começando de um ponto diferente a cada chamada para distribuir a carga. Recusas atribuíveis ao modelo da vez, como excesso de requisições ou indisponibilidade momentânea, levam à tentativa seguinte; falhas atribuíveis à conta, como credencial inválida ou ausência de crédito, interrompem imediatamente a sequência, por não serem sanáveis pela troca de modelo. Um modelo que a conta não possa utilizar é retirado da rotação até a próxima reinicialização, evitando o custo de uma requisição inútil a cada extração. O segundo nível é o fluxo de contingência: esgotada a rotação, a aplicação informa a falha e mantém o preenchimento manual disponível, garantindo que as regras de negócio operem continuamente sem comprometer a integridade dos cadastros.

## 3.19 Segurança de Dados

A segurança da informação e a proteção dos dados no Gerenciador de Eventos Acadêmicos fundamentam-se no controle de acesso, na integridade do armazenamento e na preservação da privacidade das informações trafegadas. A autenticação dos usuários é realizada por meio do padrão JSON Web Token. No momento do login, o servidor valida as credenciais informadas e emite um token assinado, o qual deve ser encaminhado no cabeçalho das requisições subsequentes para autorizar o acesso aos recursos do sistema de acordo com o perfil cadastrado. Requisições que apresentem tokens inválidos, expirados ou ausentes são sumariamente negadas pelo servidor.

Complementarmente à camada de autenticação, a aplicação aplica validações rigorosas nas regras de negócio antes de realizar qualquer inserção ou alteração na base de dados

relacional PostgreSQL. As senhas cadastradas não são salvas em texto puro: são submetidas ao algoritmo BCrypt antes da persistência (RNF-03), e a validação do tamanho considera a extensão em bytes, e não em caracteres, por ser esse o limite efetivo do algoritmo. A rotina de autenticação executa o cálculo criptográfico também quando o endereço informado não corresponde a nenhum cadastro, de modo que o tempo de resposta não revele a existência da conta. Adicionalmente, as chaves de API e os segredos de integração com serviços de inteligência artificial permanecem restritos ao ambiente do servidor.

Como a interface web e o servidor são publicados em domínios distintos, a aplicação declara explicitamente as origens autorizadas a consumir a interface de programação. Essa restrição impede que páginas de terceiros realizem requisições em nome de um usuário autenticado, complementando as verificações de autorização executadas no servidor.

Pela mesma razão, a interface declara uma política de segurança de conteúdo que relaciona nominalmente o domínio do servidor entre as origens às quais o navegador pode dirigir requisições e das quais pode carregar imagens. Os demais cabeçalhos de proteção do documento acompanham a publicação da interface.

Por fim, nenhum perfil de configuração passível de execução em produção traz segredo de assinatura embutido. Na ausência da variável correspondente, a aplicação recusa-se a iniciar e apresenta um diagnóstico explícito, comportamento preferível à inicialização com um valor conhecido publicamente, que permitiria a emissão de tokens falsos sem qualquer sinal de anormalidade.

## 4. Requisitos do Navegador/Smartphone

Para acessar o sistema, o usuário deverá utilizar um dispositivo com acesso à internet e um navegador compatível. A utilização dos recursos de inteligência artificial depende da disponibilidade e da configuração do serviço integrado. Caso esse serviço esteja indisponível, o sistema deverá permitir o cadastro manual dos eventos, garantindo a continuidade das principais funcionalidades da aplicação.

Por se tratar de uma aplicação web, o sistema não exige instalação no computador nem no smartphone do usuário: o acesso ocorre pelo navegador, a partir do endereço público da aplicação. Os requisitos abaixo valem para ambos os tipos de dispositivo:

A. Navegador atualizado: Google Chrome ou Microsoft Edge 111 ou superior, Mozilla Firefox 128 ou superior, ou Safari 16.4 ou superior, inclusive no iPhone e no iPad;

B. JavaScript habilitado, uma vez que a interface é uma aplicação de página única construída em React;

C. Armazenamento local do navegador habilitado, pois é nele que a sessão autenticada é mantida entre os acessos;

D. Leitor de PDF, nativo na maioria dos navegadores, para abrir o certificado emitido.

A interface é responsiva e se ajusta à largura da tela, de modo que as mesmas funcionalidades ficam disponíveis no smartphone sem aplicativo nativo. A preparação do ambiente para desenvolver ou executar o sistema localmente está descrita na seção 3.16.

# 5. Desenvolvimento

## 5.1 Workflow de Branches

O projeto adota um fluxo de branches baseado em Git, no qual a branch main armazena exclusivamente versões estáveis. A main é protegida: não aceita commits diretos, e toda alteração chega a ela por meio de um Pull Request aprovado por pelo menos um integrante da equipe além do autor.

Cada tarefa é desenvolvida em uma branch própria, criada a partir da main e nomeada conforme o tipo de alteração. Novas funcionalidades utilizam o prefixo feature/, como em feature/check-in-evento. Correções de problemas utilizam fix/, como em fix/fila-espera-duplicada. Melhorias estruturais que não alteram o comportamento do sistema utilizam refactor/, como em refactor/service-inscricao. Alterações na documentação utilizam docs/, como em docs/passos-instalacao.

Após a revisão e a aprovação, o Pull Request é integrado à main e a branch da tarefa é removida. Os commits seguem o padrão Conventional Commits, com os prefixos feat, fix, refactor e docs, mantendo o histórico do repositório limpo e rastreável.

## 5.2 Convenções de Códigos

O desenvolvimento do Gerenciador de Eventos Acadêmicos deverá seguir boas práticas de organização e escrita de código, tendo como referência os princípios de Clean Code, buscando manter o código simples, legível, organizado e de fácil manutenção. As classes, métodos e variáveis deverão utilizar nomes claros e relacionados às suas respectivas funções, evitando construções desnecessariamente complexas.

A organização do projeto deverá seguir a divisão em camadas definida na arquitetura do sistema, mantendo as responsabilidades de cada parte da aplicação bem delimitadas. Também serão utilizadas nomenclaturas padronizadas para as principais entidades do sistema, como Usuario, UsuarioPapel, Evento, EventoFoto, Inscricao, Presenca, Certificado, Avaliacao, ResumoAvaliacao e InteracaoIA. A comunicação entre a interface e o servidor seguirá o padrão JSON por meio da API REST, conforme definido na arquitetura.

Estabeleceu-se, ainda, que as classes de controle não acessam diretamente os repositórios e que as entidades de persistência não são expostas nas respostas da interface de programação, sendo sempre convertidas em objetos de transferência de dados. Cada regra de negócio implementada recebe, no código e no teste correspondente, a referência ao seu identificador, permitindo localizar rapidamente onde cada restrição desta documentação foi materializada.

# 6. API e Interfaces

## 6.1 Endpoints Principais

A comunicação entre a interface e o servidor ocorre por meio de uma interface de programação no estilo REST, com troca de mensagens no formato JSON. Todas as rotas são prefixadas por /api e, salvo indicação em contrário, exigem o envio do token de autenticação no cabeçalho da requisição. As datas seguem o padrão ISO 8601 e as listagens são paginadas.

### 6.1.1 Rotas de Autenticação

| Método | Rota | Perfil | Descrição |
|---|---|---|---|
| POST | /auth/registrar | Público | Cria uma nova conta de usuário (RF-01) |
| POST | /auth/login | Público | Valida as credenciais e devolve o token de acesso (RF-01) |
| GET | /auth/eu | Autenticado | Retorna os dados do usuário autenticado |

### 6.1.2 Eventos

| Método | Rota | Perfil | Descrição |
|---|---|---|---|
| GET | /eventos | Público | Lista eventos publicados, com filtros de texto e período (RF-04) |
| GET | /eventos/{id} | Público | Detalha um evento, incluindo imagens e vagas restantes |
| GET | /eventos/meus | Organizador | Lista os eventos criados pelo próprio organizador (RN-08) |
| POST | /eventos | Organizador | Cria um evento em situação de rascunho (RF-02) |
| PUT | /eventos/{id} | Organizador criador | Altera os dados do evento (RN-08 e RN-09) |
| PATCH | /eventos/{id}/status | Organizador criador | Publica, encerra ou cancela o evento |
| POST | /eventos/{id}/fotos | Organizador criador | Envia imagens de divulgação |
| DELETE | /eventos/{id}/fotos/{fotoId} | Organizador criador | Remove uma imagem de divulgação |
| POST | /eventos/interpretar | Organizador | Extrai os dados do evento a partir de texto livre (RF-03) |

A rota de interpretação não persiste o evento: devolve os campos identificados e a relação daqueles que não puderam ser extraídos, cabendo ao organizador revisar e confirmar antes da criação efetiva. Em caso de indisponibilidade do serviço de inteligência artificial, a resposta indica a falha e o preenchimento manual permanece disponível, conforme a regra RN-10.

### 6.1.3 Inscrições

| Método | Rota | Perfil | Descrição |
|---|---|---|---|
| POST | /eventos/{id}/inscricoes | Participante | Solicita vaga, confirmando ou |

| Método | Rota | Perfil | Descrição |
|---|---|---|---|
| | | | enfileirando (RF-05 e RF-06) |
| DELETE | /inscricoes/{id} | Participante titular | Cancela a própria inscrição (RF-07) |
| GET | /inscricoes/minhas | Autenticado | Lista as inscrições do usuário |
| GET | /eventos/{id}/inscricoes | Organizador criador | Lista inscritos e fila de espera |

### 6.1.4 Presença e Certificação

| Método | Rota | Perfil | Descrição |
|---|---|---|---|
| POST | /eventos/{id}/presencas | Organizador criador | Registra o check-in de um participante (RF-08) |
| GET | /eventos/{id}/presencas | Organizador criador | Lista os participantes presentes |
| POST | /inscricoes/{id}/certificado | Participante titular | Emite o certificado, se houver presença (RF-09) |
| GET | /inscricoes/{id}/certificado | Participante titular | Obtém o arquivo PDF do certificado |
| GET | /certificados/validar/{codigo} | Público | Verifica a autenticidade de um certificado |

### 6.1.5 Avaliações

| Método | Rota | Perfil | Descrição |
|---|---|---|---|
| POST | /inscricoes/{id}/avaliacao | Participante titular | Registra nota e comentário (RF-10) |
| GET | /eventos/{id}/avaliacoes | Organizador criador | Lista as avaliações recebidas |
| POST | /eventos/{id}/avaliacoes/resumo | Organizador criador | Gera o resumo por inteligência artificial (RN-07) |
| GET | /eventos/{id}/avaliacoes/resumo | Organizador criador | Consulta o último resumo gerado |

### 6.1.6 Padronização de Erros

Todas as respostas de erro seguem uma estrutura única, composta por data e hora da ocorrência, código numérico do protocolo, identificador textual do erro, mensagem explicativa, rota acessada e, quando aplicável, a relação dos campos que não passaram na validação. Essa padronização permite que a interface trate as falhas de forma uniforme e evita a exposição de detalhes internos da aplicação, em conformidade com o requisito RNF-03.

| Código | Identificador | Situação |
|---|---|---|
| 401 | CREDENCIAIS_INVALIDAS | Token ausente, expirado ou credenciais incorretas |
| 403 | ACESSO_NEGADO | Usuário autenticado sem permissão sobre o recurso (RN-08) |
| 404 | RECURSO_NAO_ENCONTRADO | Registro inexistente |
| 409 | INSCRICAO_DUPLICADA | Já existe inscrição ativa no evento (RN-03) |
| 422 | VALIDACAO | Campos obrigatórios ausentes ou fora do formato esperado |
| 422 | PRESENCA_NAO_REGISTRADA | Operação que depende de presença registrada |

| Código | Identificador | Situação |
|---|---|---|
| | | (RN-05 e RN-06) |
| 422 | REGRA_DE_NEGOCIO | Operação contrária a uma regra do sistema |
| 503 | IA_INDISPONIVEL | Serviço de inteligência artificial fora do ar (RN-10) |
| 500 | ERRO_INTERNO | Falha não prevista; a resposta não expõe detalhes internos (RNF-03) |

## 6.2 Autenticação

O controle de acesso adota o padrão JSON Web Token, escolhido por dispensar o armazenamento de sessão no servidor e, com isso, permitir que a aplicação seja executada em múltiplas instâncias sem compartilhamento de estado. O fluxo compreende quatro etapas sucessivas.

a) O usuário submete as credenciais à rota de autenticação.

b) O servidor localiza o registro pelo endereço de correio eletrônico e compara a senha informada com o resumo criptográfico armazenado. Em caso de divergência, responde com o código 401 e o identificador CREDENCIAIS_INVALIDAS, sem revelar se a falha decorreu do usuário inexistente ou da senha incorreta.

c) Confirmadas as credenciais, o servidor emite um token assinado contendo o identificador do usuário, os papéis atribuídos e o instante de expiração.

d) A interface armazena o token e o reenvia no cabeçalho de autorização de cada requisição subsequente, no formato Bearer.

A cada requisição protegida, um filtro valida a assinatura e o prazo de validade do token antes que a requisição alcance a camada de controle. Tokens ausentes, malformados, expirados ou com assinatura inválida produzem resposta 401, sem que a regra de negócio chegue a ser executada.

A autorização ocorre em dois níveis complementares. O primeiro verifica o papel exigido pela rota, impedindo, por exemplo, que um participante acesse operações privativas de organizadores. O segundo, executado na camada de serviço, confirma a propriedade do recurso: ainda que o usuário possua o papel de organizador, somente o criador do evento pode alterá-lo ou registrar presenças, conforme estabelece a regra RN-08. Essa segunda verificação não pode ser resolvida apenas por anotação de papel, razão pela qual foi deliberadamente posicionada junto à regra de negócio.

O prazo de validade do token foi fixado em oito horas, intervalo suficiente para cobrir a realização de um evento sem exigir nova autenticação do organizador durante o registro de presenças. Não há emissão de token de renovação nesta versão: expirado o prazo, o usuário realiza novo login.

# 7. Estratégia de Testes

## 7.1 Pirâmide de Testes

A estratégia de testes do Gerenciador de Eventos Acadêmicos baseia-se no conceito de Pirâmide de Testes, que organiza os testes em níveis de acordo com sua abrangência e custo de execução. Essa abordagem permite identificar falhas em diferentes etapas do desenvolvimento, garantindo maior confiabilidade e qualidade do sistema.

A base da pirâmide concentra a maior quantidade de testes, pois são testes mais rápidos e específicos, enquanto os níveis superiores possuem uma quantidade menor de testes, por validarem comportamentos mais amplos do sistema. Essa organização permite encontrar problemas inicialmente nas partes menores da aplicação, reduzindo impactos durante a utilização do sistema.

Na base da pirâmide estão os testes unitários, escritos com JUnit 5 e Mockito, responsáveis por validar métodos, classes e regras de negócio do backend em Java de forma isolada. Nesse nível verificam-se funcionalidades essenciais do sistema, como validação de dados de cadastro, controle de inscrições, gerenciamento da fila de espera, regras para registro de presença e geração de certificados.

No nível intermediário estão os testes de integração, que verificam a comunicação entre os diferentes módulos da aplicação. Eles validam o funcionamento das rotas REST, a comunicação com o banco de dados PostgreSQL e o tratamento dos dados antes do armazenamento, inclusive o fluxo de extração por saída estruturada. O banco utilizado é provisionado em contêiner pela biblioteca Testcontainers no início da execução e descartado ao final, de modo que as migrações sejam validadas a cada rodada.

No topo da pirâmide estão os testes de ponta a ponta, executados em navegador real por meio do Playwright e distribuídos em quatro arquivos de especificação, com nove verificações. Eles percorrem os fluxos principais da aplicação pela perspectiva do usuário: a navegação no catálogo, a criação de eventos pelo organizador, a inscrição de participantes com formação de fila de espera, o registro de presença, a emissão do certificado e o comportamento do sistema com a inteligência artificial indisponível.

Além disso, os testes validam os requisitos não funcionais definidos no projeto, como segurança, autorização de usuários e resiliência. Verifica-se, assim, que o sistema mantém funcionamento adequado mesmo com o serviço de inteligência artificial indisponível, permitindo o cadastro manual de eventos conforme o requisito RNF-02. A suíte do backend reúne, ao todo, 131 métodos de teste distribuídos em 20 classes, aos quais se somam as nove verificações de ponta a ponta.

## 7.2 Execução

A bateria completa é executada por meio de um único comando, tanto na máquina do desenvolvedor quanto no servidor de integração contínua, o que elimina divergências entre a verificação local e a automatizada.

```
./mvnw test                     # somente os testes unitarios
./mvnw verify                   # unitarios e de integracao (exige Docker)
npm test --prefix e2e           # ponta a ponta, com Playwright
A verificação do frontend é executada à parte, com análise estática sensível a tipos e
construção de produção:
npm run lint --prefix frontend   # ESLint com regras que usam o verificador de tipos
npm run build --prefix frontend  # executa o compilador TypeScript antes do empacotador
```

Os testes de integração provisionam automaticamente uma instância descartável de PostgreSQL, criada no início da execução e destruída ao final. Dessa forma, as migrações são validadas a cada rodada e a base de desenvolvimento permanece intocada. Nenhum teste realiza chamadas reais ao serviço de inteligência artificial: a implementação responsável pela extração de dados é substituída por uma versão de teste, o que torna o resultado determinístico, elimina custos de consumo e permite simular deliberadamente a indisponibilidade prevista no requisito RNF-02.

Concluída a execução, o relatório consolidado fica disponível no diretório de saída da construção e, nas execuções do pipeline, é publicado como artefato para consulta pela equipe. A equipe definiu cinco verificações de presença obrigatória, por incidirem sobre os pontos de maior risco do sistema:

a) Disputa pela última vaga por duas requisições simultâneas, em que uma inscrição é confirmada e a outra encaminhada à fila de espera (RN-01);

b) Cancelamento de inscrição confirmada com promoção automática do primeiro participante da fila (RN-02);

c) Tentativa de emissão de certificado sem registro de presença, que deve ser recusada (RN-05);

d) Reinscrição após cancelamento, que não pode violar a restrição de unicidade (RN-03);

e) Indisponibilidade do serviço de inteligência artificial, com manutenção do fluxo manual de cadastro (RN-10).

Os nomes dos métodos de teste referenciam o identificador da regra verificada, de modo que a leitura do relatório de execução permita associar diretamente cada falha à restrição descrita nesta documentação.

# 8. Deployment e DevOps

## 8.1 Pipeline CI/CD

O projeto adota um pipeline de integração contínua com o objetivo de verificar automaticamente cada alteração submetida ao repositório, assegurando que a branch main receba apenas código que compila corretamente e aprova a bateria de testes definida na pirâmide de testes. A automação é implementada por meio do GitHub Actions, com o fluxo descrito em um arquivo de configuração versionado junto ao código-fonte, no diretório .github/workflows.

O pipeline é acionado em duas situações: a cada envio de commits para uma branch de tarefa e a cada abertura ou atualização de Pull Request direcionado à main. Essa configuração garante que o integrante receba retorno sobre a qualidade do código antes mesmo da revisão humana, reduzindo o tempo de correção e evitando que falhas cheguem à branch estável.

### 8.1.1 Etapas do Pipeline

A execução é organizada em etapas sequenciais, de modo que a falha em qualquer uma interrompe as seguintes e marca a verificação como reprovada. A primeira etapa realiza a obtenção do código e a preparação do ambiente, com a configuração do Java na versão definida pelo projeto e o armazenamento em cache das dependências do Maven, reduzindo o tempo das execuções subsequentes.

Na sequência, é executada a compilação do projeto, que valida a integridade sintática e as dependências declaradas. Aprovada a compilação, são executados os testes unitários, responsáveis por verificar as regras de negócio isoladamente, como o controle de vagas, a fila de espera e as condições de emissão de certificado. Em seguida são executados os testes de integração, que utilizam uma instância de PostgreSQL provisionada automaticamente pelo próprio pipeline, permitindo validar as consultas, as migrações de banco e o comportamento das APIs REST em condições próximas às de execução real.

Por fim, o projeto é empacotado e o relatório de testes é publicado como artefato da execução, ficando disponível para consulta pela equipe. O fluxo contempla ainda um segundo trabalho, dedicado ao frontend, que instala as dependências a partir do arquivo de bloqueio, executa a análise estática sensível a tipos e realiza a construção de produção, na qual o compilador TypeScript é invocado antes do empacotador. Execuções simultâneas sobre a mesma referência são canceladas automaticamente, de modo que um novo envio substitua a verificação em andamento.

### 8.1.2 Critérios de Aprovação

A branch main é protegida e a aprovação do pipeline é condição obrigatória para a integração de qualquer Pull Request, somando-se à revisão por pelo menos um integrante da equipe além do autor, conforme definido no workflow de branches. Dessa forma, nenhuma alteração que apresente falha de compilação ou de testes pode ser incorporada à versão estável do sistema.

Os testes executados no pipeline não realizam chamadas reais ao serviço de inteligência artificial, utilizando uma implementação substituta do componente de extração de dados. Essa decisão evita custos de consumo da API, elimina a dependência de rede durante a verificação e torna o resultado do pipeline determinístico, além de permitir a simulação da indisponibilidade do serviço prevista no requisito RNF-02.

### 8.1.3 Tratamento de Segredos

As chaves de API, as credenciais do banco de dados e o segredo de assinatura dos tokens não são versionados no repositório. Esses valores são armazenados no cofre de segredos da plataforma de integração contínua e nas configurações de ambiente das plataformas de hospedagem, sendo injetados como variáveis durante a execução, em conformidade com o requisito RNF-03. O repositório mantém apenas um arquivo de exemplo, sem valores reais, indicando quais variáveis precisam ser configuradas.

## 8.2 Ambientes

O projeto opera com três ambientes distintos, com finalidades e bases de dados separadas. A separação impede que experimentos realizados durante o desenvolvimento alcancem dados de demonstração e assegura que a bateria de testes possa ser executada livremente, sem risco de efeito colateral.

| Ambiente | Servidor | Interface | Banco de dados |
|---|---|---|---|
| Desenvolvimento | Execução local na porta 8080 | Execução local em modo de desenvolvimento | PostgreSQL em contêiner na máquina do desenvolvedor |
| Teste automatizado | Instância efêmera criada pela bateria de testes | Não aplicável | Instância descartável criada e destruída a cada execução |
| Produção | Render | Vercel | PostgreSQL gerenciado na Neon |

Em produção, o servidor de aplicação é publicado na plataforma Render a partir da branch principal do repositório, construído com o mesmo Dockerfile utilizado localmente, e a interface web é publicada na Vercel, que reconstrói a aplicação automaticamente a cada envio aceito. A base de dados reside na Neon, serviço gerenciado de PostgreSQL, cuja cadeia de conexão é fornecida ao servidor por variável de ambiente e jamais versionada. O servidor e o banco foram provisionados na mesma região geográfica, pois cada requisição da interface origina diversas consultas ao banco, ao passo que o navegador realiza uma única chamada ao servidor; distribuí-los por regiões distintas multiplicaria a latência no trecho de maior tráfego.

Quatro características dessa composição exigem atenção na operação. A primeira é a política de suspensão por inatividade aplicada aos planos gratuitos: após cerca de quinze minutos sem requisições, a instância do servidor é suspensa, e a requisição seguinte aguarda a reinicialização do contêiner, com atraso da ordem de dezenas de segundos — motivo pelo qual se recomenda acessar a aplicação com antecedência antes de uma demonstração. A segunda é o limite de conexões simultâneas do banco gerenciado, que torna recomendável manter dimensionamento conservador do conjunto de conexões. A terceira é a necessidade de declarar o endereço da interface entre as origens autorizadas do servidor e, reciprocamente, o endereço do servidor na política de segurança de conteúdo da interface, já que as duas partes residem em domínios diferentes. A quarta é o caráter efêmero do sistema de arquivos da plataforma de hospedagem: as imagens de divulgação enviadas pelos organizadores não sobrevivem a uma nova publicação, limitação sem efeito sobre os certificados, gerados em memória a cada solicitação.

A publicação em produção permanece condicionada à aprovação prévia do pipeline. A implantação propriamente dita é acionada de forma manual nesta etapa do projeto, estratégia adequada ao escopo acadêmico por manter o controle da equipe sobre o momento da publicação, sem prejuízo da automação completa da verificação.

Duas particularidades da conexão com o banco gerenciado foram consideradas. A primeira é a exigência de transporte cifrado, declarada na própria cadeia de conexão. A segunda é a escolha do ponto de acesso direto, em detrimento do intermediário de conexões oferecido pela plataforma: este último opera em modo de transação e descarta o estado de sessão, do qual depende o bloqueio consultivo utilizado pela ferramenta de migração; além disso, a aplicação já mantém o próprio conjunto de conexões, o que torna o intermediário dispensável.

A interface e o servidor expõem, respectivamente, o endereço público da aplicação e a rota de verificação de saúde utilizada pela plataforma de hospedagem para distinguir uma instância em execução de uma instância iniciada com falha.

# 9. Performance e Otimização

## 9.1 Métricas de Eficiência

Os indicadores de eficiência foram definidos a partir dos requisitos não funcionais e dos pontos do sistema com maior probabilidade de degradação sob carga. Cada indicador possui meta explícita e método de aferição, de modo que a verificação não dependa de percepção subjetiva.

| Indicador | Meta | Origem | Forma de aferição |
|---|---|---|---|
| Tempo de carregamento da lista de eventos | Inferior a 2 segundos com 500 eventos | RNF-01 | Medição do tempo de resposta da rota de listagem com base de carga |
| Número de consultas por listagem | Constante, independentemente da quantidade de registros | Boa prática | Registro das consultas executadas durante a requisição |
| Tempo de extração por inteligência artificial | Até 20 segundos, com interrupção após esse limite | RN-10 | Tempo registrado na tabela de interações |
| Tempo de geração do certificado | Inferior a 3 segundos | RF-09 | Medição do tempo de resposta da rota de emissão |
| Tempo de resposta das demais rotas | Inferior a 500 milissegundos no percentil 95 | Boa prática | Métricas expostas pelo próprio servidor de aplicação |
| Conexões simultâneas ao banco | Dentro do limite do plano contratado | Restrição da plataforma | Monitoramento do conjunto de conexões |

A principal medida de otimização adotada é estrutural e antecede qualquer ajuste de infraestrutura: os índices definidos na seção 3.14 sustentam diretamente as duas consultas mais frequentes do sistema, a listagem pública de eventos e a localização do próximo participante da fila de espera. Sem eles, ambas degradariam de forma proporcional ao volume de registros.

A segunda medida diz respeito ao carregamento de associações. Todas as associações entre entidades foram declaradas com carregamento tardio, e as consultas que exibem dados relacionados utilizam junções explícitas. Essa decisão previne o padrão conhecido como consulta N+1, no qual a listagem de registros dispara uma consulta adicional por item retornado, e que constitui a causa mais comum de violação de metas de tempo de resposta em aplicações com mapeamento objeto-relacional.

Deliberadamente, não foi adotada camada de cache nesta versão. A medida só se justificaria caso as metas acima deixassem de ser atendidas com os índices e o carregamento adequadamente configurados, e sua introdução prematura acrescentaria complexidade de invalidação sem benefício mensurável no volume de dados previsto para o projeto.

# 10. Troubleshooting

## 10.1 Problemas Comuns

A relação a seguir reúne as falhas de maior probabilidade de ocorrência durante o desenvolvimento e a operação do sistema, com a respectiva causa provável e o procedimento de correção. Ela foi construída a partir das características específicas da arquitetura adotada e das plataformas de hospedagem escolhidas.

| Sintoma | Causa provável | Procedimento |
|---|---|---|
| A aplicação não inicia e o erro menciona validação de migração | Script já aplicado foi alterado após a execução, invalidando a soma de verificação registrada pelo Flyway | Em desenvolvimento, recriar a base com docker compose down -v seguido de docker compose up -d. Nunca editar um script já integrado à main: criar um novo script numerado |
| Rota pública responde 401 | Caminho não declarado entre as rotas liberadas na configuração de segurança | Incluir o caminho na lista de rotas públicas. As versões recentes do Spring Security adotam padrões mais restritivos, exigindo declaração explícita |
| A interface recebe erro de origem bloqueada | Endereço da interface ausente entre as origens autorizadas do servidor | Declarar o domínio de produção e o endereço local nas origens permitidas e reiniciar o servidor |
| Primeira requisição do dia demora dezenas de segundos | Suspensão por inatividade aplicada ao plano gratuito da plataforma de hospedagem | Comportamento esperado do plano. Para demonstrações, acessar a aplicação com antecedência para reativá-la |
| Erro de limite de conexões ao banco | Conjunto de conexões da aplicação dimensionado acima do limite do plano | Reduzir o número máximo de conexões na configuração da aplicação |
| Cadastro por texto responde 503 | Serviço de linguagem indisponível, sem crédito ou com chave inválida | Comportamento previsto na regra RN-10. Verificar a chave e o saldo; o cadastro manual permanece disponível |
| A extração devolve campos vazios ou incorretos | Texto de origem não contém a informação, ou o modelo não a identificou | Os campos não identificados são devolvidos em lista própria para preenchimento manual. O sistema não deve inferir valores ausentes |
| Inscrição confirmada além do limite de vagas | Verificação de vagas executada fora da transação ou sem controle de concorrência | Confirmar que a verificação e a gravação ocorrem na mesma transação e que o controle de versão da entidade Evento está ativo (RN-01) |
| Certificado emitido sem presença registrada | Verificação de presença ausente ou posicionada após a geração do arquivo | A consulta ao registro de presença deve anteceder a geração do PDF, conforme o diagrama de interação da seção 3.7 (RN-05) |
| Testes passam localmente e falham no pipeline | Dependência de dados remanescentes na base local | Os testes de integração devem partir de base limpa. Verificar se há dados criados por um teste e consumidos por outro |
| A aplicação não inicia e o erro menciona segredo de assinatura ausente ou curto | Variável JWT_SECRET não definida no ambiente ou no arquivo local de configuração | Definir a variável com valor aleatório de no mínimo 32 bytes. Nenhum perfil de produção traz valor padrão, e a recusa de iniciar é deliberada: subir com segredo conhecido permitiria a emissão de tokens falsos sem sinal de anormalidade |
| A plataforma de hospedagem | Aplicação fixada em porta | A porta do servidor é lida da variável PORT, com |

| Sintoma | Causa provável | Procedimento |
|---|---|---|
| acusa ausência de porta em escuta | própria, enquanto a plataforma atribui a porta por variável de ambiente | 8080 como valor padrão. Não definir essa variável manualmente na plataforma |
| A interface carrega, mas nenhuma requisição chega ao servidor, e o console do navegador menciona política de segurança de conteúdo | Domínio do servidor ausente nas diretivas de conexão e de imagem declaradas pela interface | Incluir o endereço do servidor na política de segurança de conteúdo da interface e publicar novamente. O bloqueio ocorre no navegador, antes do envio, e não produz registro no servidor |
| O cadastro por texto responde com indisponibilidade, e o registro indica modelo inexistente | Identificador de modelo retirado do catálogo do provedor ou indisponível para a conta | Comportamento previsto: a rotação segue para o modelo seguinte e retira o indisponível da sequência até a próxima reinicialização. Atualizar a lista de modelos quando o registro apontar recusas recorrentes |
| Falha na migração ao conectar no banco gerenciado | Uso do ponto de acesso intermediário, que descarta o estado de sessão exigido pelo bloqueio consultivo da ferramenta de migração | Utilizar a cadeia de conexão direta, sem o intermediário de conexões, mantendo a exigência de transporte cifrado |

## 11. Referências e Recursos

As referências abaixo reúnem a documentação oficial das tecnologias empregadas e as obras consultadas na definição das práticas de desenvolvimento adotadas pelo projeto.


ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação — referências — elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.

BECK, K. Test-Driven Development: By Example. Boston: Addison-Wesley, 2002.

FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

MARTIN, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008.

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language (OMG UML), versão 2.5.1. Disponível em: https://www.omg.org/spec/UML/

INTERNET ENGINEERING TASK FORCE. RFC 7519: JSON Web Token (JWT). Disponível em: https://datatracker.ietf.org/doc/html/rfc7519

SPRING. Spring Boot Reference Documentation. Disponível em: https://docs.spring.io/spring-boot/

SPRING. Spring AI Reference Documentation. Disponível em: https://docs.spring.io/spring-ai/reference/

SPRING. Spring Security Reference Documentation. Disponível em: https://docs.spring.io/spring-security/reference/

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Disponível em: https://www.postgresql.org/docs/

REDGATE. Flyway Documentation. Disponível em: https://documentation.red-gate.com/flyway

APRYSE. iText Core Documentation. Disponível em: https://itextpdf.com/resources/api-documentation

CONVENTIONAL COMMITS. Conventional Commits 1.0.0. Disponível em: https://www.conventionalcommits.org/

RENDER. Render Documentation. Disponível em: https://render.com/docs

VERCEL. Vercel Documentation. Disponível em: https://vercel.com/docs

NEON. Neon Documentation. Disponível em: https://neon.com/docs

GROQ. GroqCloud Documentation. Disponível em: https://console.groq.com/docs

MICROSOFT. Playwright Documentation. Disponível em: https://playwright.dev/docs/intro

TESTCONTAINERS. Testcontainers for Java. Disponível em: https://java.testcontainers.org/

META. React Documentation. Disponível em: https://react.dev/

TAILWIND LABS. Tailwind CSS Documentation. Disponível em: https://tailwindcss.com/docs

# 12. Apêndices

## Apêndice A — Variáveis de Ambiente

As variáveis abaixo devem ser definidas no arquivo local de configuração durante o desenvolvimento e nas configurações de ambiente das plataformas de hospedagem em produção. Nenhuma delas é versionada no repositório, que mantém apenas o arquivo de exemplo sem valores reais.

| Variável | Obrigatória | Finalidade |
|---|---|---|
| DB_URL | Sim | Cadeia de conexão JDBC com o banco de dados, incluindo a exigência de transporte cifrado em produção |
| DB_USER | Sim | Usuário do banco de dados |
| DB_PASSWORD | Sim | Senha do banco de dados |
| JWT_SECRET | Sim | Segredo utilizado na assinatura dos tokens de acesso |
| APP_URL | Sim | Endereço público da aplicação, usado na montagem do link de validação do certificado |
| CORS_ORIGENS | Sim | Origens autorizadas a consumir a interface de programação; aceita curinga de porta e de subdomínio |
| OPENAI_API_KEY | Não | Chave da OpenAI, utilizada apenas no perfil correspondente |
| IA_HABILITADA | Não | Permite desativar a integração com inteligência artificial sem alterar o código |
| SPRING_PROFILES_ACTIVE | Não | Perfil de configuração ativo. Na ausência, vale o perfil de desenvolvimento |
| GROQ_API_KEY | Sim, no perfil groq | Chave do provedor de inteligência artificial adotado em produção |
| GROQ_MODELOS | Não | Lista de modelos percorridos em rotação, separados por vírgula. Na ausência, vale a lista padrão do perfil |
| TZ | Não | Fuso horário da aplicação, do qual dependem a janela de check-in (RN-04) e a verificação de evento encerrado (RN-06) |
| PORT | Não | Porta de escuta do servidor, atribuída pela plataforma de hospedagem. Localmente, 8080 |
| SWAGGER_HABILITADO | Não | Exposição da documentação interativa da interface de programação, desativada por padrão |
| VITE_API_URL | Sim, na interface | Endereço público do servidor, incorporado à construção do frontend |

## Apêndice B — Glossário

| Termo | Definição | Onde aparece |
|---|---|---|
| Saída estruturada | Recurso que anexa à requisição o esquema do objeto esperado, fazendo o modelo de linguagem devolver dados em estrutura definida previamente, em vez de texto livre | Seções 2.2 e 3.18 |
| Fila de espera | Ordenação dos participantes que solicitaram vaga em evento já lotado, com promoção automática em caso de desistência | RN-01 e RN-02 |
| Código de autenticidade | Identificador único impresso no certificado, que permite a qualquer pessoa verificar sua validade | RN-05 e seção 6.1.4 |
| Migração | Script versionado que cria ou altera a estrutura do banco de dados de forma reproduzível | Seção 3.14 |
| Barra de ativação | Elemento do diagrama de sequência que indica o período em que um objeto está em execução | Seções 3.5 e 3.7 |
| Pull Request | Solicitação de integração de uma branch à principal, sujeita a revisão e à aprovação do pipeline | Seções 5.1 e 8.1 |
| Carregamento tardio | Estratégia em que dados associados só são buscados no banco quando efetivamente utilizados | Seção 9.1 |
| Rotação de modelos | Percurso por uma lista de modelos de linguagem até que um responda, iniciando de ponto diferente a cada chamada para distribuir a carga entre eles | Seções 3.18 e 10.1 |
| Política de segurança de conteúdo | Conjunto de regras declarado pela interface que limita as origens das quais o navegador pode carregar recursos e às quais pode dirigir requisições | Seções 3.19 e 8.2 |